

Міністерство освіти та науки України

Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»

Факультет інформатики та обчислювальної техніки

Кафедра інформатики та програмної інженерії

Звіт до комп’ютерного практикуму №5

з дисципліни «Комп’ютерне моделювання дискретно-подійних систем»

Прийняв:
Викладач
Д.С. Коринецький

Виконав
Студент 4 курсу, гр. ІП-33
Ломакін Максим

Оцінка: _____

«___» _____ 2026 р.

Київ 2026

Формалізація дискретно-подійних систем стохастичною мережею Петрі

1. Текст завдання:

Розробити стохастичну мережу Петрі для кожної з наведених задач.

1. (20 балів) Конвеєрна система складається з п’ятьох обслуговуючих пристроїв, розташованих уздовж стрічки конвеєра. Деталі надходять на опрацювання на перший пристрій із постійною швидкістю, рівної 4 одиниці за 1 хвилину. Тривалість обслуговування на кожному пристрої розподілена за експоненціальним законом з математичним сподіванням 1 хвилина. Вільного місця перед кожним конвеєром немає, тому пристрій може зняти деталь із конвеєра, тільки якщо знаходиться в стані «вільний». Якщо перший пристрій вільний, то деталь обробляється на ньому. По закінченні обробляння деталь залишає систему. Якщо перший пристрій зайнятий у момент надходження деталі, деталь по конвеєру надходить до другого пристрою. Інтервал проходження деталі між пристроями дорівнює 1 хвилина. Якщо при прямуванні деталі по конвеєру всі пристрої були зайняті, вона повертається до першого пристрою з затримкою 5 хвилин. Метою моделювання є визначення статистичних характеристик часу перебування деталі в системі, завантаження обслуговуючих пристроїв і кількості зайнятих пристроїв.
2. (25 балів) Експериментальна роботизована гнучка виробнича система має два верстати із числовим пультом керування, три роботи, пункт прибуття і склад оброблених деталей. Деталі прибувають на пункт прибуття кожні 40 секунд згідно з експоненціальним законом розподілу, захоплюються одним з вільних роботів і переміщуються ним до першого верстата, після чого робот звільняється. Після завершення обробки на першому верстаті деталь захоплюється одним з роботів і переміщується на другий верстат, а після обробки на другому верстаті – одним з роботів переміщується на склад оброблених деталей. Кожний з верстатів може одночасно обробляти до трьох деталей. Час переміщення робота між пунктом прибуття та першим верстатом, першим і другим верстатом, другим верстатом та пунктом зберігання оброблених деталей складає відповідно 6, 7, і 5 секунд незалежно від того, холостий це хід, чи ні. Роботу потрібний час 8±1 секунд на захоплення або вивільнення деталей. Час

обробки на першому верстаті розподілений за нормальним законом із середнім значення 60 секунд і стандартним відхиленням 10 секунд. Середній час обробки на другому верстаті дорівнює 100 секунд і має експоненціальний закон розподілу. Метою моделювання є визначення найкращого (з точки зору підвищення пропускної здатності гнучкої виробничої системи) способу закріплення роботів до операцій. Можливі варіанти закріплення: 1) по одному роботу на кожний з трьох шляхів переміщення деталей (пункт прибуття – перший верстат, перший верстат – другий верстат, другий верстат, склад); 2) кожний робот може використовуватися на кожному шляху переміщення деталей (при цьому повинен займатися найближчий з роботів).

3. (25 балів) На маршруті приміського сполучення працюють два мікроавтобуси (А і В), кожний з яких має n місць. Мікроавтобус А користується більшою популярністю, ніж автобус В, оскільки водій мікроавтобуса А їздить акуратніше і швидше. Тому пасажир, який підійшов до зупинки, сідає в мікроавтобус В тільки у випадку, коли автобуса А немає. Мікроавтобус відправляється на маршрут, якщо всі місця в ньому зайняті. Пасажири підходять до зупинки через 0,5±0,2 хвилин і, якщо немає мікроавтобусів, утворюють чергу. Якщо черга більша, ніж 30 осіб, то пасажир не стає у чергу і йде до іншого маршруту. Припускається, що всі пасажири їдуть до кінця маршруту. На проходження маршруту мікроавтобус А витрачає 20±5 хвилин, а мікроавтобус В – 30±5 хвилин. Після того, як пасажири звільнили автобус (протягом часу 5±1 хвилин), він їде у зворотному напрямку тим же чином. Плата за проїзд складає 20 гривень. Авто підприємство стільки ж втрачає (недоотримує), якщо пасажир, прийшовши на зупинку, не стає у чергу і обирає інший маршрут. Метою моделювання є визначення таких характеристик: час очікування пасажира у черзі; кількість місць n (не більше 25), при якому час очікування в черзі пасажира буде мінімальним; виручку автопідприємства за день від маршруту, якщо мікроавтобуси працюють 10 годин на добу.
4. (30 балів) У супермаркеті планується ввести систему управління запасами холодильників. Час між надходженнями замовлень на холодильники має експоненціальний розподіл з математичним

сподіванням 0,2 тижні. Якщо покупцю знадобився холодильник тоді, коли його в запасі немає, він у 80 % випадків відправляється в інший найближчий магазин, представляючи тим самим продаж, що не відбувся для даного універмагу. У 20 % таких випадків робиться повторне замовлення, і покупці чекають надходження наступної партії вантажу. Магазин використовує періодичну систему перегляду стана запасів, у якому запас проглядається кожні 4 тижні і приймається рішення про необхідність здійснення замовлення. Стратегія прийняття рішення складається в розміщенні замовлення, що доводить запас до контрольного рівня, що складає 72 холодильники. Поточний стан запасу визначається як наявний запас плюс замовлені раніше приймачі і мінус невдоволений попит. Якщо поточний стан запасів менше або дорівнює 18 холодильникам (точка замовлення), здійснюється розміщення замовлення. Час доставки (час між розміщенням замовлення і його одержання) постійний і складає 3 тижні. Початкові умови: стан запасу – 72 холодильника, невдоволеного попиту немає. Визначити середню кількість холодильників у запасі, середній час між продажами, що не здійснилися.

2. Код програми:

Програму написано мовою Python 3.12. Вихідний код, результати та опис побудованих мереж розміщено в репозиторії https://github.com/kqjqxb/Computer-simulation (каталог `lab5`). Призначення файлів:

- `fun_rand.py` – генератори випадкових величин (експоненційний, рівномірний, нормальний);
- `petri.py` – стохастична часова мережа Петрі: позиції, переходи із затримкою, пріоритетом і ймовірністю, багатоканальні переходи, інформаційні дуги та кратності дуг, алгоритм імітації, статистика, опис мережі;
- `runner.py` – серії прогонів і довірчі інтервали;
- `task1_conveyor.py`, `task2_fms.py`, `task3_minibus.py`, `task4_inventory.py` – мережі Петрі завдань 1–4, експерименти та перевірка;
- `plots.py` – графіки результатів.

Файл fun_rand.py

```
import math
import random

_rng = random.Random()


def seed(value):
    _rng.seed(value)


def exp(time_mean):
    a = 0.0
    while a == 0.0:
        a = _rng.random()
    return -time_mean * math.log(a)


def unif(time_min, time_max):
    a = 0.0
    while a == 0.0:
        a = _rng.random()
    return time_min + a * (time_max - time_min)


def norm(time_mean, time_deviation):
    return _rng.gauss(time_mean, time_deviation)


def uniform01():
    return _rng.random()
```

Файл petri.py

```
"""Стохастична часова мережа Петрі.

Позиції містять маркери, переходи мають часову затримку, пріоритет і ймовірність спрацювання.
Переходи багатоканальні: у переході одночасно може перебувати кілька маркерів, для кожного з яких
запам'ятовується момент виходу. Вхідна дуга може бути інформаційною – вона перевіряє наявність маркерів
у позиції, але не забирає їх. Кожна дуга має кратність.

Маркер зберігає момент свого створення. Вихідна дуга з параметром carry переносить у вихідну позицію
маркери, взяті з указаної вхідної позиції, інші вихідні дуги створюють нові маркери. Так моделюється
рух вимоги мережею і вимірюється час між подіями з однією й тією самою вимогою. Позиція видає маркери
в порядку надходження, а маркери, що виходять з переходів одночасно, виводяться в порядку їх входу
в переходи.
"""
import heapq
import math
from collections import deque

import fun_rand


class Delay:
    """Затримка переходу: const – стала mean, exp – експоненційна із середнім mean,
    unif – рівномірна на [mean - dev, mean + dev], norm – нормальна (mean, dev), обмежена знизу нулем."""

    KINDS = ("const", "exp", "unif", "norm")

    def __init__(self, kind="const", mean=0.0, dev=0.0):
        if kind not in self.KINDS:
            raise ValueError(kind)
        self.kind, self.mean, self.dev = kind, float(mean), float(dev)

    def __call__(self):
        if self.kind == "exp":
            return fun_rand.exp(self.mean)
        if self.kind == "unif":
            return fun_rand.unif(self.mean - self.dev, self.mean + self.dev)
        if self.kind == "norm":
            return max(0.0, fun_rand.norm(self.mean, self.dev))
        return self.mean

    def __str__(self):
        m, d = f"{self.mean:g}", f"{self.dev:g}"
        return {"const": m, "exp": f"exp({m})", "unif": f"{m} ± {d}", "norm": f"N({m}; {d})"}[self.kind]
```

```
    def spec(self):
        return {"distribution": self.kind, "mean": self.mean, "deviation": self.dev}


def const(mean):
    return Delay("const", mean)


def exp(mean):
    return Delay("exp", mean)


def unif(mean, dev):
    return Delay("unif", mean, dev)


def norm(mean, dev):
    return Delay("norm", mean, dev)


class Token:
    __slots__ = ("born",)

    def __init__(self, born):
        self.born = born


class Place:
    def __init__(self, name, mark=0):
        self.name, self.initial = name, mark
        self.tokens = deque(Token(0.0) for _ in range(mark))
        self.dependents = []  # переходи, для яких позиція є вхідною
        self.area = 0.0
        self.last = self.start = 0.0
        self.max_mark = self.min_mark = mark

    @property
    def mark(self):
        return len(self.tokens)

    def _touch(self, t):
        self.area += len(self.tokens) * (t - self.last)
        self.last = t

    def put(self, t, tokens):
        self._touch(t)
        self.tokens.extend(tokens)
        self.max_mark = max(self.max_mark, len(self.tokens))

    def take(self, t, n):
        self._touch(t)
        taken = [self.tokens.popleft() for _ in range(n)]
        self.min_mark = min(self.min_mark, len(self.tokens))
        return taken

    def reset_statistics(self, t):
        self._touch(t)
        self.area, self.start = 0.0, t
        self.max_mark = self.min_mark = len(self.tokens)

    def mean(self, t):
        self._touch(t)
        return self.area / (t - self.start) if t > self.start else float(len(self.tokens))


class Transition:
    def __init__(self, name, delay, priority=0, probability=1.0, channels=math.inf):
        self.name = name
        self.delay = delay if isinstance(delay, Delay) else const(delay)
        self.priority, self.probability, self.channels = priority, probability, channels
        self.inputs = []  # (позиція, кратність, інформаційна)
        self.outputs = []  # (позиція, кратність, ім'я вхідної позиції, з якої переносяться маркери)
        self.buffer = []  # купа (момент виходу, номер входу, {позиція: взяті маркери})
        self.on_out = None  # on_out(t, consumed) – спостерігач виходу маркера з переходу
        self.entered = self.fired = 0
        self.area = 0.0
        self.last = self.start = 0.0
```

```
        self.max_busy = 0

    def enabled(self):
        if len(self.buffer) >= self.channels:
            return False
        return all(len(p.tokens) >= n for p, n, _ in self.inputs)

    @property
    def next_time(self):
        return self.buffer[0][0] if self.buffer else math.inf

    def _touch(self, t):
        self.area += len(self.buffer) * (t - self.last)
        self.last = t

    def act_in(self, t, number):
        consumed = {p.name: p.take(t, n) for p, n, info in self.inputs if not info}
        self._touch(t)
        heapq.heappush(self.buffer, (t + self.delay(), number, consumed))
        self.entered += 1
        self.max_busy = max(self.max_busy, len(self.buffer))

    def act_out(self, t):
        self._touch(t)
        _, _, consumed = heapq.heappop(self.buffer)
        for place, n, carry in self.outputs:
            moved = consumed.get(carry, [])[:n]
            place.put(t, moved + [Token(t) for _ in range(n - len(moved))])
        self.fired += 1
        if self.on_out:
            self.on_out(t, consumed)

    def reset_statistics(self, t):
        self._touch(t)
        self.area, self.start = 0.0, t
        self.entered = self.fired = 0
        self.max_busy = len(self.buffer)

    def mean_busy(self, t):
        self._touch(t)
        return self.area / (t - self.start) if t > self.start else float(len(self.buffer))


class Watch:
    """Середнє за часом довільної функції стану мережі та її розподіл за часом."""

    def __init__(self, fn):
        self.fn = fn
        self.area = self.span = 0.0
        self.hist = {}

    def add(self, net, dt):
        if dt > 0:
            value = self.fn(net)
            self.area += value * dt
            self.span += dt
            self.hist[value] = self.hist.get(value, 0.0) + dt

    def reset_statistics(self):
        self.area = self.span = 0.0
        self.hist = {}


def _grouped(names):
    """Список імен з об'єднанням однакових сусідніх: «Доставка ×72»."""
    groups = []
    for name in names:
        if groups and groups[-1][0] == name:
            groups[-1][1] += 1
        else:
            groups.append([name, 1])
    return ", ".join(name if n == 1 else f"{name} ×{n}" for name, n in groups)


class PetriNet:
    def __init__(self, name):
        self.name = name
```

```
        self.places, self.transitions, self.watches = {}, {}, {}
        self.time = 0.0
        self.stats_start = 0.0
        self.events = 0

    def place(self, name, mark=0):
        self.places[name] = Place(name, mark)
        return self.places[name]

    def transition(self, name, delay=0.0, inputs=None, outputs=None, info=None, carry=None, priority=0,
                   probability=1.0, channels=math.inf):
        """inputs, info, outputs – словники {позиція: кратність дуги}; carry – {вихідна позиція: вхідна}."""
        tr = Transition(name, delay, priority, probability, channels)
        carry = carry or {}
        for arcs, is_info in ((inputs or {}, False), (info or {}, True)):
            for pname, n in arcs.items():
                place = self.places[pname]
                tr.inputs.append((place, n, is_info))
                place.dependents.append(tr)
        for pname, n in (outputs or {}).items():
            tr.outputs.append((self.places[pname], n, carry.get(pname)))
        self.transitions[name] = tr
        return tr

    def watch(self, name, fn):
        self.watches[name] = Watch(fn)

    def mark(self, name):
        return len(self.places[name].tokens)

    def _fire_inputs(self, t, dirty, enabled, entered):
        """Вхід маркерів у переходи: поки є дозволені переходи, спрацьовує перехід з найвищим
        пріоритетом; конфлікт переходів з однаковим пріоритетом розв'язується за ймовірностями."""
        while True:
            for tr in dirty:
                if tr.enabled():
                    enabled.add(tr)
                else:
                    enabled.discard(tr)
            dirty.clear()
            if not enabled:
                return
            top = max(tr.priority for tr in enabled)
            group = sorted((tr for tr in enabled if tr.priority == top), key=self._order.get)
            chosen = group[0]
            if len(group) > 1:
                r = fun_rand.uniform01() * sum(tr.probability for tr in group)
                for chosen in group:
                    r -= chosen.probability
                    if r < 0:
                        break
            self._number += 1
            chosen.act_in(t, self._number)
            entered.append(chosen.name)
            dirty.add(chosen)
            for place, _, info in chosen.inputs:
                if not info:
                    dirty.update(place.dependents)

    def _advance(self, t_new, warmup):
        t_old = self.time
        if t_old < warmup <= t_new and not self._reset_done:
            for w in self.watches.values():
                w.add(self, warmup - t_old)
            self.reset_statistics(warmup)
            t_old = warmup
        for w in self.watches.values():
            w.add(self, t_new - t_old)
        self.time = t_new

    def reset_statistics(self, t):
        self._reset_done = True
        self.stats_start = t
        for obj in (*self.places.values(), *self.transitions.values()):
            obj.reset_statistics(t)
        for w in self.watches.values():
            w.reset_statistics()
```

```
def simulate(self, time, warmup=0.0, trace=0, hooks=(), out=print):
    """Імітація до моменту time. hooks – функції hook(net), що викликаються після кожної події;
    trace – кількість перших подій, для яких виводиться стан мережі."""
    self._order = {tr: i for i, tr in enumerate(self.transitions.values())}
    self._number = 0
    self._reset_done = warmup <= 0.0
    dirty, enabled = set(self.transitions.values()), set()
    entered, fired = [], []
    while True:
        self._fire_inputs(self.time, dirty, enabled, entered)
        for hook in hooks:
            hook(self)
        if self.events < trace:
            self._print_state(fired, entered, out)
        entered, fired = [], []
        tnext = min(tr.next_time for tr in self.transitions.values())
        if tnext > time:
            self._advance(time, warmup)
            break
        self._advance(tnext, warmup)
        ready = [tr for tr in self.transitions.values() if tr.next_time == tnext]
        while ready:
            # одночасні виходи – у порядку входу маркерів у переходи
            tr = min(ready, key=lambda x: x.buffer[0][1])
            tr.act_out(tnext)
            fired.append(tr.name)
            dirty.add(tr)
            for place, _, _ in tr.outputs:
                dirty.update(place.dependents)
            if tr.next_time != tnext:
                ready.remove(tr)
        self.events += 1
    return self.results()

def _print_state(self, fired, entered, out):
    lines = [f"t = {self.time:.4f}"]
    if fired:
        lines.append("  вихід з переходів: " + _grouped(fired))
    if entered:
        lines.append("  вхід у переходи:   " + _grouped(entered))
    marks = [f"{p.name}={p.mark}" for p in self.places.values() if p.mark]
    busy = [f"{tr.name}={len(tr.buffer)} (вихід {tr.next_time:.3f})" for tr in self.transitions.values()
            if tr.buffer]
    lines.append("  маркування: " + ", ".join(marks))
    lines.append("  у переходах: " + ", ".join(busy))
    out("\n".join(lines) + "\n")

def results(self):
    t = self.time
    return {
        "time": t - self.stats_start,
        "places": {p.name: {"mean": p.mean(t), "min": p.min_mark, "max": p.max_mark, "final": p.mark}
                   for p in self.places.values()},
        "transitions": {tr.name: {"fired": tr.fired, "entered": tr.entered, "mean_busy": tr.mean_busy(t),
                                  "max_busy": tr.max_busy} for tr in self.transitions.values()},
        "watches": {name: {"mean": w.area / w.span if w.span else math.nan,
                           "distribution": {v: s / w.span for v, s in sorted(w.hist.items())}}
                    for name, w in self.watches.items()},
    }

def describe(self):
    """Опис структури мережі (для відтворення в інших реалізаціях)."""
    return {
        "name": self.name,
        "places": [{"name": p.name, "mark": p.initial} for p in self.places.values()],
        "transitions": [{
            "name": tr.name, **tr.delay.spec(), "priority": tr.priority, "probability": tr.probability,
            "channels": None if tr.channels == math.inf else tr.channels,
            "inputs": [{"place": p.name, "n": n} for p, n, info in tr.inputs if not info],
            "info_inputs": [{"place": p.name, "n": n} for p, n, info in tr.inputs if info],
            "outputs": [{"place": p.name, "n": n, **({"carry": c} if c else {})}
                        for p, n, c in tr.outputs],
        } for tr in self.transitions.values()],
    }
```

Файл runner.py

```
import statistics

from scipy import stats

import fun_rand


def replicate(run, seeds):
    """Виконує run() для кожного зерна генератора; повертає список результатів прогонів."""
    results = []
    for s in seeds:
        fun_rand.seed(s)
        results.append(run())
    return results


def summary(values):
    """Середнє та півширина 95 % довірчого інтервалу за t-розподілом Стьюдента."""
    values = list(values)
    mean = statistics.fmean(values)
    if len(values) < 2:
        return mean, 0.0
    return mean, stats.t.ppf(0.975, len(values) - 1) * statistics.stdev(values) / len(values) ** 0.5


def fmt(mean_ci, nd=4):
    mean, ci = mean_ci
    return f"{mean:.{nd}f} ± {ci:.{nd}f}"
```

Файл task1_conveyor.py

```
"""Завдання 1: конвеєрна система з п'ятьма обслуговуючими пристроями."""
import json
import statistics
from pathlib import Path

import fun_rand
from petri import PetriNet, const, exp
from runner import fmt, replicate, summary

RESULTS = Path(__file__).resolve().parent / "results"
DEVICES = 5
ARRIVAL, SERVICE_MEAN, STEP, BACK = 0.25, 1.0, 1.0, 5.0  # хвилини
TIME, WARMUP, SEEDS = 10000.0, 1000.0, range(1, 21)
TAIL = 300.0  # час перебування рахується для деталей, що надійшли не пізніше TIME − TAIL


def build():
    net = PetriNet("Конвеєрна система")
    net.place("Генератор", 1)
    for i in range(1, DEVICES + 1):
        net.place(f"Деталь біля П{i}")
        net.place(f"П{i} вільний", 1)
    net.place("Оброблені деталі")
    net.transition("Надходження", const(ARRIVAL), inputs={"Генератор": 1},
                   outputs={"Генератор": 1, "Деталь біля П1": 1})
    for i in range(1, DEVICES + 1):
        part, free = f"Деталь біля П{i}", f"П{i} вільний"
        net.transition(f"Обробка П{i}", exp(SERVICE_MEAN), inputs={part: 1, free: 1},
                       outputs={free: 1, "Оброблені деталі": 1}, carry={"Оброблені деталі": part}, priority=1)
        if i < DEVICES:
            name, delay, target = f"Рух до П{i + 1}", STEP, f"Деталь біля П{i + 1}"
        else:
            name, delay, target = "Повернення до П1", BACK, "Деталь біля П1"
        net.transition(name, const(delay), inputs={part: 1}, outputs={target: 1}, carry={target: part})
    net.watch("Зайняті пристрої", busy_devices)
    return net


def busy_devices(net):
    return sum(1 - net.mark(f"П{i} вільний") for i in range(1, DEVICES + 1))


def in_system(net):
```

```
    """Деталі в системі: на конвеєрі (у переходах руху та повернення) і на обробці."""
    return sum(len(tr.buffer) for name, tr in net.transitions.items() if name != "Надходження") \
        + sum(net.mark(f"Деталь біля П{i}") for i in range(1, DEVICES + 1))


def run(warmup=WARMUP, hooks=(), trace=0, out=print):
    net = build()
    times = []

    def record(t, consumed):
        token = next(iter(consumed.values()))[0]
        if warmup <= token.born <= TIME - TAIL:
            times.append(t - token.born)

    for i in range(1, DEVICES + 1):
        net.transitions[f"Обробка П{i}"].on_out = record
    r = net.simulate(TIME, warmup=warmup, hooks=hooks, trace=trace, out=out)
    tr = r["transitions"]
    served = sum(tr[f"Обробка П{i}"]["fired"] for i in range(1, DEVICES + 1))
    in_sys = sum(v["mean_busy"] for k, v in tr.items() if k != "Надходження") \
        + sum(r["places"][f"Деталь біля П{i}"]["mean"] for i in range(1, DEVICES + 1))
    return {
        "times": times,
        "mean_time": statistics.fmean(times),
        "load": [tr[f"Обробка П{i}"]["mean_busy"] for i in range(1, DEVICES + 1)],
        "share": [tr[f"Обробка П{i}"]["fired"] / served for i in range(1, DEVICES + 1)],
        "busy": r["watches"]["Зайняті пристрої"]["mean"],
        "busy_dist": r["watches"]["Зайняті пристрої"]["distribution"],
        "throughput": served / r["time"],
        "returns_per_part": tr["Повернення до П1"]["fired"] / served,
        "in_system": in_sys,
    }


class Invariants:
    """Перевірка P-інваріантів після кожної події та підрахунок найбільшої кількості деталей у системі."""

    def __init__(self):
        self.checked, self.max_in_system = 0, 0

    def __call__(self, net):
        for i in range(1, DEVICES + 1):
            assert net.mark(f"П{i} вільний") + len(net.transitions[f"Обробка П{i}"].buffer) == 1
        assert net.mark("Генератор") + len(net.transitions["Надходження"].buffer) == 1
        created = net.transitions["Надходження"].fired
        assert created == net.mark("Оброблені деталі") + in_system(net)
        self.max_in_system = max(self.max_in_system, in_system(net))
        self.checked += 1


def histogram(times, width=0.5):
    counts = {}
    for t in times:
        k = int(t // width)
        counts[k] = counts.get(k, 0) + 1
    return {"width": width, "counts": [counts.get(k, 0) for k in range(max(counts) + 1)]}


def main():
    net = build()
    with open(RESULTS / "net1_conveyor.json", "w", encoding="utf-8") as f:
        json.dump(net.describe(), f, ensure_ascii=False, indent=1)
    print(f"Конвеєр: надходження кожні {ARRIVAL} хв, обробка exp({SERVICE_MEAN}), рух між пристроями "
          f"{STEP} хв, повернення {BACK} хв, {DEVICES} пристроїв\n")
    print("Трасування перших подій:\n")
    fun_rand.seed(1)
    run(warmup=0.0, trace=12, out=print)

    fun_rand.seed(100)
    check = Invariants()
    one = run(warmup=0.0, hooks=[check], out=print)
    print(f"Перевірка інваріантів: {check.checked} подій, порушень немає; найбільше деталей у системі "
          f"{check.max_in_system}")

    runs = replicate(run, SEEDS)
    times = [t for r in runs for t in r["times"]]
    q = statistics.quantiles(times, n=20)
```

```
    table = [("середній час перебування, хв", summary(r["mean_time"] for r in runs))]
    table += [(f"завантаження П{i + 1}", summary(r["load"][i] for r in runs)) for i in range(DEVICES)]
    table += [
        ("середня кількість зайнятих пристроїв", summary(r["busy"] for r in runs)),
        ("пропускна здатність, деталей/хв", summary(r["throughput"] for r in runs)),
        ("повернень до П1 на одну деталь", summary(r["returns_per_part"] for r in runs)),
        ("середня кількість деталей у системі", summary(r["in_system"] for r in runs)),
    ]
    dist = {k: statistics.fmean(r["busy_dist"].get(k, 0.0) for r in runs) for k in range(DEVICES + 1)}
    share = [statistics.fmean(r["share"][i] for r in runs) for i in range(DEVICES)]
    time_stats = {"mean": statistics.fmean(times), "stdev": statistics.stdev(times), "min": min(times),
                  "median": statistics.median(times), "p90": q[17], "p95": q[18], "max": max(times),
                  "count": len(times)}

    print(f"\nРЕЗУЛЬТАТИ ({len(SEEDS)} прогонів по {TIME:.0f} хв, розгін {WARMUP:.0f} хв, "
          f"95 % довірчий інтервал):")
    for label, value in table:
        print(f"  {label:40} {fmt(value)}")
    print("\nЧас перебування деталі (усі прогони): "
          + ", ".join(f"{k} = {v:.4f}" for k, v in time_stats.items() if k != "count")
          + f", деталей {len(times)}")
    print("Розподіл кількості зайнятих пристроїв: " + ", ".join(f"P({k}) = {p:.4f}" for k, p in dist.items()))
    print("Частка деталей, оброблених на пристрої: "
          + ", ".join(f"П{i + 1} {s:.4f}" for i, s in enumerate(share)))

    lam = statistics.fmean(r["throughput"] for r in runs)
    w = table[0][1][0]
    print(f"\nПеревірка: пропускна здатність {lam:.4f} при інтенсивності надходження {1 / ARRIVAL:.1f}; "
          f"сума завантажень {sum(v[0] for _, v in table[1:6]):.4f}, "
          f"λ·E[S] = {1 / ARRIVAL * SERVICE_MEAN:.1f}")
    print(f"Формула Літтла: λ·W = {lam:.4f} · {w:.4f} = {lam * w:.4f}, середня кількість деталей у системі "
          f"{table[-1][1][0]:.4f}")

    with open(RESULTS / "task1.json", "w", encoding="utf-8") as f:
        json.dump({"table": [{"metric": m, "mean": v[0], "ci": v[1]} for m, v in table],
                   "time_stats": time_stats,
                   "busy_distribution": dist, "share": share, "throughput": lam, "little": lam * w,
                   "check_events": check.checked, "max_in_system": check.max_in_system,
                   "single_mean_time": one["mean_time"],
                   "hist": histogram(times)}, f, ensure_ascii=False, indent=1)


if __name__ == "__main__":
    main()
```

Файл task2_fms.py

```
"""Завдання 2: роботизована гнучка виробнича система - два верстати, три роботи."""
import json
import statistics
from pathlib import Path

import fun_rand
from petri import PetriNet, const, exp, norm, unif
from runner import fmt, replicate, summary

RESULTS = Path(__file__).resolve().parent / "results"
ARRIVAL_MEAN = 40.0  # с
HANDLE = unif(8, 1)  # захоплення або вивільнення деталі
MACHINE_SLOTS = 3
POSITION = {"вхід": 0.0, "В1": 6.0, "В2": 13.0, "склад": 18.0}  # точки вздовж лінії переміщення, с ходу
# шлях: (назва, звідки, куди, позиція із запитом, позиція місць на верстаті призначення,
#         позиція місць, що звільняються після захоплення, позиція, куди потрапляє деталь)
ROUTES = [
    ("вхід-В1", "вхід", "В1", "Деталь на вході", "Вільні місця В1", None, "Деталь на В1"),
    ("В1-В2", "В1", "В2", "Оброблена на В1", "Вільні місця В2", "Вільні місця В1", "Деталь на В2"),
    ("В2-склад", "В2", "склад", "Оброблена на В2", None, "Вільні місця В2", "Склад"),
]
TIME, WARMUP, SEEDS = 100000.0, 10000.0, range(1, 21)
SATURATION = 10 ** 5  # початковий запас деталей на вході в режимі необмеженого надходження


def distance(a, b):
    return abs(POSITION[a] - POSITION[b])


def build(variant, arrival_mean=ARRIVAL_MEAN):
```

```
    """variant 1 – по одному роботу на кожен шлях; variant 2 – будь-який робот на будь-якому шляху,
    виконувати переміщення починає найближчий вільний робот. arrival_mean=None – необмежене надходження."""
    net = PetriNet(f"Гнучка виробнича система, варіант {variant}")
    net.place("Генератор", 1 if arrival_mean else 0)
    net.place("Деталь на вході", 0 if arrival_mean else SATURATION)
    for m in ("B1", "B2"):
        net.place(f"Вільні місця {m}", MACHINE_SLOTS)
        net.place(f"Деталь на {m}")
        net.place(f"Оброблена на {m}")
    net.place("Склад")
    for route, *_ in ROUTES:
        for stage in ("Робот біля деталі", "Деталь захоплена", "Деталь доставлена"):
            net.place(f"{stage} ({route})")
    if variant == 1:
        for k in range(1, 4):
            net.place(f"Робот {k} вільний", 1)
            net.place(f"Робот {k} повертається")
    else:
        for loc in POSITION:
            net.place(f"Вільний робот: {loc}", int(loc != "склад"))

    if arrival_mean:
        net.transition("Надходження", exp(arrival_mean), inputs={"Генератор": 1},
                       outputs={"Генератор": 1, "Деталь на вході": 1})
    net.transition("Обробка на B1", norm(60, 10), inputs={"Деталь на B1": 1}, outputs={"Оброблена на B1": 1},
                   carry={"Оброблена на B1": "Деталь на B1"})
    net.transition("Обробка на B2", exp(100), inputs={"Деталь на B2": 1}, outputs={"Оброблена на B2": 1},
                   carry={"Оброблена на B2": "Деталь на B2"})

    for k, (route, origin, target, request, slot, freed, result) in enumerate(ROUTES, start=1):
        at_part, held, delivered = (f"{s} ({route})" for s in ("Робот біля деталі", "Деталь захоплена",
                                                               "Деталь доставлена"))
        # робота призначають, лише якщо на верстаті призначення є вільне місце; місце резервується
        inputs = {request: 1, **({slot: 1} if slot else {})}
        if variant == 1:
            net.transition(f"Призначення робота {k}", inputs={**inputs, f"Робот {k} вільний": 1},
                           outputs={at_part: 1}, carry={at_part: request})
        else:
            for loc in POSITION:
                d = distance(loc, origin)
                net.transition(f"Виклик робота: {loc} → {route}", const(d),
                               inputs={**inputs, f"Вільний робот: {loc}": 1}, outputs={at_part: 1},
                               carry={at_part: request}, priority=int(100 - d))
        net.transition(f"Захоплення ({route})", HANDLE, inputs={at_part: 1},
                       outputs={held: 1, **({freed: 1} if freed else {})}, carry={held: at_part})
        net.transition(f"Переміщення ({route})", const(distance(origin, target)), inputs={held: 1},
                       outputs={delivered: 1}, carry={delivered: held})
        robot = f"Робот {k} повертається" if variant == 1 else f"Вільний робот: {target}"
        net.transition(f"Вивільнення ({route})", HANDLE, inputs={delivered: 1},
                       outputs={result: 1, robot: 1}, carry={result: delivered})
        if variant == 1:
            net.transition(f"Повернення робота {k}", const(distance(origin, target)),
                           inputs={f"Робот {k} повертається": 1}, outputs={f"Робот {k} вільний": 1})
    return net


def robots_busy(net):
    """Кількість зайнятих роботів (з урахуванням холостого ходу)."""
    if "Робот 1 вільний" in net.places:
        return 3 - sum(net.mark(f"Робот {k} вільний") for k in range(1, 4))
    return 3 - sum(net.mark(f"Вільний робот: {loc}") for loc in POSITION)


def inside(net, prefix, suffix=""):
    """Кількість маркерів у переходах, назва яких починається з prefix і закінчується suffix."""
    return sum(len(t.buffer) for name, t in net.transitions.items()
               if name.startswith(prefix) and name.endswith(suffix))


def check_invariants(net):
    """P-інваріанти роботів і місць на верстатах; перевіряються в моменти, коли в переходах немає
    маркерів з нульовою затримкою."""
    if any(t.next_time == net.time for t in net.transitions.values()):
        return
    on_route = {route: sum(net.mark(f"{s} ({route})") for s in ("Робот біля деталі", "Деталь захоплена",
                                                                "Деталь доставлена"))
                + sum(inside(net, f"{s} ({route})") for s in ("Захоплення", "Переміщення", "Вивільнення"))
```

```
                for route, *_ in ROUTES}
    if "Робот 1 вільний" in net.places:
        for k, (route, *_) in enumerate(ROUTES, start=1):
            robot = net.mark(f"Робот {k} вільний") + net.mark(f"Робот {k} повертається")
            assert robot + inside(net, f"Повернення робота {k}") + on_route[route] == 1
    else:
        free = sum(net.mark(f"Вільний робот: {loc}") for loc in POSITION)
        assert free + inside(net, "Виклик") + sum(on_route.values()) == 3
    # місце на верстаті зайняте від призначення робота, який везе деталь, до захоплення її наступним роботом
    for m, into, out in (("B1", "вхід-B1", "B1-B2"), ("B2", "B1-B2", "B2-склад")):
        on_machine = net.mark(f"Деталь на {m}") + inside(net, f"Обробка на {m}") \
            + net.mark(f"Оброблена на {m}")
        before_grab = net.mark(f"Робот біля деталі ({out})") + inside(net, f"Захоплення ({out})") \
            + inside(net, "Виклик", out)
        reserved = inside(net, "Виклик", into) + on_route[into]
        assert net.mark(f"Вільні місця {m}") + reserved + on_machine + before_grab == MACHINE_SLOTS


def run(variant, arrival_mean=ARRIVAL_MEAN, time=TIME, warmup=WARMUP, hooks=(), trace=0, out=print,
        tail=5000.0):
    net = build(variant, arrival_mean)
    times = []

    def record(t, consumed):
        born = next(iter(consumed.values()))[0].born
        if warmup <= born <= time - tail:
            times.append(t - born)

    net.transitions["Вивільнення (B2-склад)"].on_out = record
    net.watch("Зайняті роботи", robots_busy)
    r = net.simulate(time, warmup=warmup, hooks=hooks, trace=trace, out=out)
    tr, pl = r["transitions"], r["places"]
    hour = r["time"] / 3600
    result = {
        "throughput": tr["Вивільнення (B2-склад)"]["fired"] / hour,
        "arrivals": tr["Надходження"]["fired"] / hour if arrival_mean else None,
        "time_in_system": statistics.fmean(times) if times else None,
        "input_queue": pl["Деталь на вході"]["mean"],
        "input_queue_final": pl["Деталь на вході"]["final"],
        "robots_busy": r["watches"]["Зайняті роботи"]["mean"],
        "m1_slots": MACHINE_SLOTS - pl["Вільні місця B1"]["mean"],
        "m2_slots": MACHINE_SLOTS - pl["Вільні місця B2"]["mean"],
        "m1_processing": tr["Обробка на B1"]["mean_busy"],
        "m2_processing": tr["Обробка на B2"]["mean_busy"],
        "m1_done_waiting": pl["Оброблена на B1"]["mean"],
        "m2_done_waiting": pl["Оброблена на B2"]["mean"],
        "idle_moves": sum(v["mean_busy"] for k, v in tr.items() if k.startswith(("Виклик", "Повернення"))),
    }
    if variant == 1:
        result["robot_load"] = [1 - pl[f"Робот {k} вільний"]["mean"] for k in range(1, 4)]
    else:
        result["calls"] = {k: v["fired"] for k, v in tr.items() if k.startswith("Виклик")}
    return result


INTERVALS = (45.0, 50.0, 55.0, 60.0, 70.0, 80.0)
METRICS = [
    ("throughput", "пропускна здатність, деталей/год"),
    ("time_in_system", "середній час перебування деталі, с"),
    ("input_queue", "середня черга на вході"),
    ("robots_busy", "середня кількість зайнятих роботів"),
    ("m1_slots", "зайняті місця B1 (з резервом)"),
    ("m2_slots", "зайняті місця B2 (з резервом)"),
    ("m1_processing", "деталей в обробці на B1"),
    ("m2_processing", "деталей в обробці на B2"),
    ("m1_done_waiting", "оброблені на B1 чекають робота"),
    ("m2_done_waiting", "оброблені на B2 чекають робота"),
    ("idle_moves", "роботів у холостому ході"),
]


def far_call(state):
    """У стані мережі є вхід у перехід виклику робота, що стоїть не на початку шляху."""
    for line in state.splitlines():
        if line.strip().startswith("вхід у переходи"):
            for name in line.split(":", 1)[1].split(", "):
                if name.strip().startswith("Виклик робота"):
```

```
                    loc, route = name.strip().removeprefix("Виклик робота: ").split(" → ")
                    if not route.startswith(loc):
                        return True
    return False


def series(variant, arrival_mean, keys):
    runs = replicate(lambda: run(variant, arrival_mean), SEEDS)
    return {k: summary(r[k] for r in runs) for k in keys if runs[0][k] is not None}, runs


def main():
    for variant in (1, 2):
        with open(RESULTS / f"net2_fms_variant{variant}.json", "w", encoding="utf-8") as f:
            json.dump(build(variant).describe(), f, ensure_ascii=False, indent=1)
    print(f"ТВС: надходження exp({ARRIVAL_MEAN:g}) с, захоплення/вивільнення {HANDLE} с, B1 N(60; 10) с, "
          f"B2 exp(100) с, по {MACHINE_SLOTS} місця на верстаті; ходи вхід-B1 6 с, B1-B2 7 с, B2-склад 5 с\n")
    for variant, events in ((1, 9), (2, 9)):
        print(f"Трасування перших подій, варіант {variant}:\n")
        fun_rand.seed(4)
        run(variant, trace=events)
    print("Варіант 2, перший виклик робота, що стоїть не на початку шляху (холостий хід):\n")
    fun_rand.seed(4)
    lines = []
    run(2, time=3000.0, warmup=0.0, trace=10 ** 6, out=lines.append)
    first = next(i for i, s in enumerate(lines) if far_call(s))
    print("\n".join(lines[first - 1:first + 2]))

    for variant in (1, 2):
        fun_rand.seed(100 + variant)
        checked = []
        run(variant, hooks=[check_invariants, lambda n: checked.append(1)])
        print(f"Перевірка інваріантів, варіант {variant}: {len(checked)} подій, порушень немає")

    base, capacity, sweep = {}, {}, {}
    for variant in (1, 2):
        base[variant], runs = series(variant, ARRIVAL_MEAN, [k for k, _ in METRICS] + ["arrivals"])
        if variant == 1:
            base[variant]["robot_load"] = [summary(r["robot_load"][k] for r in runs) for k in range(3)]
        else:
            calls = {k: statistics.fmean(r["calls"][k] for r in runs) for k in runs[0]["calls"]}
            base[variant]["calls"] = calls
        capacity[variant], _ = series(variant, None, [k for k, _ in METRICS])
        sweep[variant] = {a: series(variant, a, ["throughput", "time_in_system", "input_queue"])[0]
                          for a in INTERVALS}

    print(f"\nРЕЗУЛЬТАТИ при надходженні кожні {ARRIVAL_MEAN:g} с ({len(SEEDS)} прогонів по {TIME:.0f} с, "
          f"розгін {WARMUP:.0f} с, 95 % довірчий інтервал):")
    print(f"  {'показник':40} {'варіант 1':>20} {'варіант 2':>20}")
    print(f"  {'інтенсивність надходження, деталей/год':40} {fmt(base[1]['arrivals'], 2):>20} "
          f"{fmt(base[2]['arrivals'], 2):>20}")
    for key, label in METRICS:
        print(f"  {label:40} {fmt(base[1][key], 2):>20} {fmt(base[2][key], 2):>20}")
    print("  завантаження роботів 1-3 (варіант 1): " + ", ".join(fmt(v, 3) for v in base[1]["robot_load"]))

    print("\nНеобмежене надходження деталей (пропускна здатність системи):")
    for key, label in METRICS:
        if key in capacity[1]:
            print(f"  {label:40} {fmt(capacity[1][key], 2):>20} {fmt(capacity[2][key], 2):>20}")

    print("\nЗалежність від середнього інтервалу надходження:")
    print(f"  {'інтервал, с':>11} {'вар. 1, дет./год':>17} {'вар. 2, дет./год':>17} {'вар. 1: час, с':>18} "
          f"{'вар. 2: час, с':>18}")
    for a in INTERVALS:
        s1, s2 = sweep[1][a], sweep[2][a]
        print(f"  {a:11.0f} {fmt(s1['throughput'], 2):>17} {fmt(s2['throughput'], 2):>17} "
              f"{fmt(s1['time_in_system'], 0):>18} {fmt(s2['time_in_system'], 0):>18}")

    calls = base[2]["calls"]
    total = sum(calls.values())
    print("\nВаріант 2, частка викликів робота за місцем, де він стояв:")
    for name, count in sorted(calls.items(), key=lambda x: -x[1]):
        if count:
            print(f"  {name:32} {count / total:.4f}")
    by_route = {}
    for name, count in calls.items():
        loc, route = name.removeprefix("Виклик робота: ").split(" → ")
```

```
        origin = next(r[1] for r in ROUTES if r[0] == route)
        n, d = by_route.get(route, (0.0, 0.0))
        by_route[route] = (n + count, d + count * distance(loc, origin))
    idle = {route: d / n for route, (n, d) in by_route.items()}
    print("  середній холостий хід до деталі, с: " + ", ".join(f"{r} {v:.2f}" for r, v in idle.items()))

    x = capacity[1]["throughput"][0] / 3600
    print("\nПеревірка (варіант 1, необмежене надходження): за пропускної здатності X = "
          f"{capacity[1]['throughput'][0]:.2f} дет./год")
    print(f"  в обробці на B1 X·60 = {x * 60:.3f} (імітація {capacity[1]['m1_processing'][0]:.3f}), "
          f"на B2 X·100 = {x * 100:.3f} (імітація {capacity[1]['m2_processing'][0]:.3f})")
    cycle = [2 * 8 + 2 * distance(r[1], r[2]) for r in ROUTES]
    print(f"  зайняті роботи X·(28 + 30 + 26) = {x * sum(cycle):.3f} "
          f"(імітація {capacity[1]['robots_busy'][0]:.3f})")

    with open(RESULTS / "task2.json", "w", encoding="utf-8") as f:
        json.dump({"base": base, "capacity": capacity, "sweep": {v: {str(a): s for a, s in d.items()}
                                                                  for v, d in sweep.items()},
                   "idle": idle, "metrics": METRICS}, f, ensure_ascii=False, indent=1)


if __name__ == "__main__":
    main()
```

Файл task3_minibus.py

```
"""Завдання 3: два мікроавтобуси на маршруті приміського сполучення."""
import json
import statistics
from pathlib import Path

import fun_rand
from petri import PetriNet, unif
from runner import fmt, replicate, summary

RESULTS = Path(__file__).resolve().parent / "results"
STOPS = (1, 2)
BUSES = {"A": unif(20, 5), "B": unif(30, 5)}  # час проходження маршруту, хв
ARRIVAL, UNLOAD = unif(0.5, 0.2), unif(5, 1)
QUEUE_LIMIT, FARE = 30, 20
DAY, SEEDS = 600.0, range(1, 51)  # 10 годин роботи
START = {"A": 1, "B": 2}  # на початку дня А стоїть на зупинці 1, В – на зупинці 2
SEATS = range(1, 26)
SEATS_NOTE = ("опис наведено для n = 1; n – кількість місць: кратність дуг «Пасажири в … → Рейс …», "
              "«Рейс … → Перевезені пасажири», «Висадка … → Вільні місця …» і початкове маркування "
              "«Вільні місця А на З1», «Вільні місця В на З2»")


def other(s):
    return 3 - s


def build(n):
    net = PetriNet(f"Маршрут мікроавтобусів, n = {n}")
    for s in STOPS:
        net.place(f"Генератор З{s}", 1)
        net.place(f"Пасажир на З{s}")
        net.place(f"Черга на З{s}")
        net.place(f"Місця в черзі З{s}", QUEUE_LIMIT)
        net.place(f"Пішли з З{s}")
    for b in BUSES:
        for s in STOPS:
            here = START[b] == s
            net.place(f"{b} на З{s}", int(here))
            net.place(f"Вільні місця {b} на З{s}", n if here else 0)
            net.place(f"Пасажири в {b} на З{s}")
            net.place(f"{b} прибув на З{s}")
    net.place("Перевезені пасажири")

    for s in STOPS:
        passenger, queue, room = f"Пасажир на З{s}", f"Черга на З{s}", f"Місця в черзі З{s}"
        net.transition(f"Прихід на З{s}", ARRIVAL, inputs={f"Генератор З{s}": 1},
                       outputs={f"Генератор З{s}": 1, passenger: 1})
        # пасажир сідає в В, лише коли на зупинці немає А: пріоритет посадки в А вищий
        for b, bonus in (("A", 1), ("B", 0)):
            seats, inside = f"Вільні місця {b} на З{s}", f"Пасажири в {b} на З{s}"
            net.transition(f"Посадка з черги в {b} на З{s}", inputs={queue: 1, seats: 1},
```

```
                        outputs={inside: 1, room: 1}, carry={inside: queue}, priority=5 + bonus)
            net.transition(f"Посадка в {b} на З{s}", inputs={passenger: 1, seats: 1}, outputs={inside: 1},
                        carry={inside: passenger}, priority=3 + bonus)
        net.transition(f"Стати в чергу на З{s}", inputs={passenger: 1, room: 1}, outputs={queue: 1},
                       carry={queue: passenger}, priority=2)
        net.transition(f"Відмова на З{s}", inputs={passenger: 1}, outputs={f"Пішли з З{s}": 1}, priority=1)
    for b, trip in BUSES.items():
        for s in STOPS:
            t = other(s)
            net.transition(f"Рейс {b} З{s}→З{t}", trip,
                        inputs={f"{b} на З{s}": 1, f"Пасажири в {b} на З{s}": n},
                        outputs={f"{b} прибув на З{t}": 1, "Перевезені пасажири": n},
                        carry={"Перевезені пасажири": f"Пасажири в {b} на З{s}"})
            net.transition(f"Висадка {b} на З{t}", UNLOAD, inputs={f"{b} прибув на З{t}": 1},
                        outputs={f"{b} на З{t}": 1, f"Вільні місця {b} на З{t}": n})
    return net


def stable(net):
    """У переходах немає маркерів, що виходять у поточний момент (миттєві переходи завершені)."""
    return all(tr.next_time > net.time for tr in net.transitions.values())


class Invariants:
    def __init__(self, n):
        self.n, self.checked = n, 0

    def __call__(self, net):
        if not stable(net):
            return
        tr = net.transitions
        for b in BUSES:
            where = [net.mark(f"{b} на З{s}") + net.mark(f"{b} прибув на З{s}")
                     + len(tr[f"Рейс {b} З{s}→З{other(s)}"].buffer) + len(tr[f"Висадка {b} на З{s}"].buffer)
                     for s in STOPS]
            assert sum(where) == 1
            for s in STOPS:
                seats = net.mark(f"Вільні місця {b} на З{s}") + net.mark(f"Пасажири в {b} на З{s}")
                assert seats == self.n * net.mark(f"{b} на З{s}")
        for s in STOPS:
            assert net.mark(f"Черга на З{s}") + net.mark(f"Місця в черзі З{s}") == QUEUE_LIMIT
            if net.mark(f"Черга на З{s}"):
                assert not any(net.mark(f"{b} на З{s}") for b in BUSES)
            arrived = tr[f"Прихід на З{s}"].fired
            boarded = sum(tr[f"{k} {b} на З{s}"].fired for b in BUSES
                          for k in ("Посадка з черги в", "Посадка в"))
            assert arrived == boarded + tr[f"Відмова на З{s}"].fired + net.mark(f"Черга на З{s}")
        self.checked += 1


def run(n, time=DAY, hooks=(), trace=0, out=print):
    net = build(n)
    waits, queued = [], []

    def from_queue(t, consumed):
        w = t - next(iter(consumed.values()))[0].born
        waits.append(w)
        queued.append(w)

    for s in STOPS:
        for b in BUSES:
            net.transitions[f"Посадка з черги в {b} на З{s}"].on_out = from_queue
            net.transitions[f"Посадка в {b} на З{s}"].on_out = lambda t, consumed: waits.append(0.0)
    r = net.simulate(time, hooks=hooks, trace=trace, out=out)
    tr = r["transitions"]
    boarded = len(waits)
    lost = sum(tr[f"Відмова на З{s}"]["fired"] for s in STOPS)
    arrived = sum(tr[f"Прихід на З{s}"]["fired"] for s in STOPS)
    joined = sum(tr[f"Стати в чергу на З{s}"]["fired"] for s in STOPS)
    return {
        "wait": statistics.fmean(waits),
        "wait_queued": statistics.fmean(queued) if queued else 0.0,
        "queue": statistics.fmean(r["places"][f"Черга на З{s}"]["mean"] for s in STOPS),
        "boarded": boarded, "lost": lost, "arrived": arrived, "joined": joined,
        "revenue": FARE * (boarded - lost),
        "income": FARE * boarded, "loss": FARE * lost,
        "trips": {b: sum(tr[f"Рейс {b} З{s}→З{other(s)}"]["entered"] for s in STOPS) for b in BUSES},
```

```
        "time": r["time"],
    }


def main():
    with open(RESULTS / "net3_minibus.json", "w", encoding="utf-8") as f:
        json.dump(build(1).describe() | {"parameter": SEATS_NOTE}, f, ensure_ascii=False, indent=1)
    print(f"Мікроавтобуси: пасажири кожні {ARRIVAL} хв на кожну з двох кінцевих зупинок, "
          f"черга до {QUEUE_LIMIT}, "
          f"рейс A {BUSES['A']} хв, B {BUSES['B']} хв, висадка {UNLOAD} хв, день {DAY:.0f} хв\n")
    print("Трасування перших подій (n = 3):\n")
    fun_rand.seed(1)
    run(3, trace=14, out=print)

    fun_rand.seed(100)
    check = Invariants(10)
    run(10, hooks=[check])
    print(f"Перевірка інваріантів (n = 10): {check.checked} стійких станів, порушень немає")

    # довгий прогін для перевірки формули Літтла та пропускної здатності
    fun_rand.seed(200)
    long_time = 100 * DAY
    lr = run(25, time=long_time)
    lam = lr["joined"] / long_time / len(STOPS)
    print(f"Довгий прогін {long_time:.0f} хв, n = 25: середня черга на зупинці {lr['queue']:.4f}, "
          f"λ_черги · W_черги = {lam:.4f} · {lr['wait_queued']:.4f} = {lam * lr['wait_queued']:.4f}")
    cap = 25 * (1 / (2 * 20 + 2 * 5) + 1 / (2 * 30 + 2 * 5))
    print(f"Перевезено за хв в одному напрямку {lr['boarded'] / long_time / 2:.4f}; "
          f"оцінка при постійній черзі "
          f"n·(1/50 + 1/70) = {cap:.4f}; рейсів A {lr['trips']['A']}, B {lr['trips']['B']} "
          f"(оцінка {long_time / 25:.0f} і {long_time / 35:.0f})")

    sweep = []
    print(f"\nРЕЗУЛЬТАТИ ({len(SEEDS)} днів для кожного n, 95 % довірчий інтервал):")
    print(f"  {'n':>2} {'очікування, хв':>18} {'серед тих, хто чекав':>22} {'перевезено':>10} "
          f"{'пішли':>8} {'виручка, грн':>20}")
    for n in SEATS:
        runs = replicate(lambda: run(n), SEEDS)
        row = {"n": n, **{k: summary(r[k] for r in runs) for k in
                          ("wait", "wait_queued", "queue", "boarded", "lost", "revenue", "income", "loss")},
               "trips_A": statistics.fmean(r["trips"]["A"] for r in runs),
               "trips_B": statistics.fmean(r["trips"]["B"] for r in runs)}
        sweep.append(row)
        print(f"  {n:2d} {fmt(row['wait'], 2):>18} {fmt(row['wait_queued'], 2):>22} "
              f"{row['boarded'][0]:10.1f} {row['lost'][0]:8.1f} {fmt(row['revenue'], 0):>20}")
    best = min(sweep, key=lambda r: r["wait"][0])
    print(f"\nНайменший середній час очікування при n = {best['n']}: {fmt(best['wait'], 3)} хв")
    print(f"Виручка за день при n = {best['n']}: дохід {best['income'][0]:.0f}, "
          f"втрати {best['loss'][0]:.0f}, "
          f"виручка {fmt(best['revenue'], 0)} грн")

    with open(RESULTS / "task3.json", "w", encoding="utf-8") as f:
        json.dump({"sweep": sweep, "best": best["n"], "check_states": check.checked,
                   "little": {"queue": lr["queue"], "rate": lam, "wait": lr["wait_queued"]},
                   "capacity": {"sim": lr["boarded"] / long_time / 2, "estimate": cap, "trips": lr["trips"],
                                "time": long_time}}, f, ensure_ascii=False, indent=1)


if __name__ == "__main__":
    main()
```

Файл task4_inventory.py

```
"""Завдання 4: управління запасами холодильників з періодичним переглядом запасу."""
import json
import math
import statistics
from pathlib import Path

import fun_rand
from petri import PetriNet, const, exp
from runner import fmt, replicate, summary

RESULTS = Path(__file__).resolve().parent / "results"
DEMAND_MEAN = 0.2  # тижні між замовленнями покупців
LOST_PROBABILITY = 0.8
REVIEW, LEAD = 4.0, 3.0
```

```
LEVEL, REORDER = 72, 18
TIME, WARMUP, SEEDS = 5000.0, 52.0, range(1, 101)


def build():
    net = PetriNet("Управління запасами")
    for name, mark in (("Генератор попиту", 1), ("Покупець", 0), ("Запас", LEVEL), ("Продано", 0),
                       ("Відкладений попит", 0), ("Втрачені продажі", 0), ("Нестача до рівня", 0),
                       ("Таймер перегляду", 1), ("Перегляд", 0), ("Формування замовлення", 0)):
        net.place(name, mark)
    net.transition("Надходження покупця", exp(DEMAND_MEAN), inputs={"Генератор попиту": 1},
                   outputs={"Генератор попиту": 1, "Покупець": 1})
    net.transition("Продаж", inputs={"Покупець": 1, "Запас": 1},
                   outputs={"Продано": 1, "Нестача до рівня": 1},
                   priority=3)
    net.transition("Відмова від покупки", inputs={"Покупець": 1}, outputs={"Втрачені продажі": 1}, priority=1,
                   probability=LOST_PROBABILITY)
    net.transition("Повторне замовлення", inputs={"Покупець": 1},
                   outputs={"Відкладений попит": 1, "Нестача до рівня": 1}, priority=1,
                   probability=1 - LOST_PROBABILITY)
    net.transition("Видача відкладеного", inputs={"Запас": 1, "Відкладений попит": 1}, outputs={"Продано": 1},
                   priority=4)
    net.transition("Перегляд запасу", const(REVIEW), inputs={"Таймер перегляду": 1},
                   outputs={"Таймер перегляду": 1, "Перегляд": 1})
    # поточний стан запасу ≤ 18  ⇔  нестача до контрольного рівня 72 ≥ 54
    net.transition("Рішення замовити", inputs={"Перегляд": 1}, info={"Нестача до рівня": LEVEL - REORDER},
                   outputs={"Формування замовлення": 1}, priority=2)
    net.transition("Без замовлення", inputs={"Перегляд": 1})
    # кожен холодильник нестачі замовляється окремим маркером; усі маркери входять у перехід одночасно
    # і одночасно виходять через 3 тижні – це одна партія
    net.transition("Доставка", const(LEAD), inputs={"Нестача до рівня": 1}, info={"Формування замовлення": 1},
                   outputs={"Запас": 1}, priority=1)
    net.transition("Кінець замовлення", inputs={"Формування замовлення": 1})
    return net


def position(net):
    """Поточний стан запасу: наявний запас + замовлені раніше – невдоволений попит."""
    return net.mark("Запас") + len(net.transitions["Доставка"].buffer) - net.mark("Відкладений попит")


def check_invariant(net):
    """P-інваріант з урахуванням маркерів усередині переходів: перехід «Продаж» уже забрав холодильник
    із запасу, але ще не додав одиницю нестачі."""
    inside = len(net.transitions["Продаж"].buffer)
    assert position(net) + inside + net.mark("Нестача до рівня") == LEVEL


def run(warmup=WARMUP, time=TIME, hooks=(), trace=0, out=print):
    net = build()
    lost, orders = [], []

    def moments(target):
        """Спостерігач виходу з переходу: запам'ятовує моменти після розгону."""
        return lambda t, consumed: target.append(t) if t >= warmup else None

    net.transitions["Відмова від покупки"].on_out = moments(lost)
    net.transitions["Рішення замовити"].on_out = moments(orders)
    net.watch("Немає в запасі", lambda n: n.mark("Запас") == 0)
    net.watch("Поточний стан запасу", position)
    r = net.simulate(time, warmup=warmup, hooks=hooks, trace=trace, out=out)
    tr = r["transitions"]
    demand = tr["Надходження покупця"]["fired"]
    gaps = [b - a for a, b in zip(lost, lost[1:])]
    return {
        "stock": r["places"]["Запас"]["mean"],
        "lost_interval": statistics.fmean(gaps) if gaps else math.nan,
        "lost_rate": len(lost) / r["time"],
        "lost_share": len(lost) / demand,
        "backorder_share": tr["Повторне замовлення"]["fired"] / demand,
        "backorders": r["places"]["Відкладений попит"]["mean"],
        "stockout_time": r["watches"]["Немає в запасі"]["mean"],
        "position": r["watches"]["Поточний стан запасу"]["mean"],
        "orders": len(orders),
        "order_size": tr["Доставка"]["entered"] / max(len(orders), 1),
        "demand_rate": demand / r["time"],
        "sold_rate": (tr["Продаж"]["fired"] + tr["Видача відкладеного"]["fired"]) / r["time"],
```

```
        "lost_gaps": gaps,
    }


def direct(time=TIME, warmup=WARMUP):
    """Незалежна перевірка: та сама політика управління запасами, змодельована подієво без мережі Петрі."""
    stock, backlog, on_order, arrival = LEVEL, 0, 0, math.inf
    t, next_demand, next_review = 0.0, fun_rand.exp(DEMAND_MEAN), REVIEW
    area, lost = 0.0, []
    while True:
        t_next = min(next_demand, next_review, arrival, time)
        if t_next > warmup:
            area += stock * (t_next - max(t, warmup))
        t = t_next
        if t >= time:
            break
        if t == arrival:
            stock += on_order
            on_order, arrival = 0, math.inf
            served = min(stock, backlog)
            stock, backlog = stock - served, backlog - served
        elif t == next_review:
            next_review += REVIEW
            ip = stock + on_order - backlog
            if ip <= REORDER:
                on_order, arrival = LEVEL - ip, t + LEAD
        else:
            next_demand = t + fun_rand.exp(DEMAND_MEAN)
            if stock > 0:
                stock -= 1
            elif fun_rand.uniform01() < LOST_PROBABILITY:
                if t >= warmup:
                    lost.append(t)
            else:
                backlog += 1
    gaps = [b - a for a, b in zip(lost, lost[1:])]
    return {"stock": area / (time - warmup), "lost_interval": statistics.fmean(gaps)}


def trajectory(weeks=60.0):
    points = []
    run(warmup=0.0, time=weeks, hooks=[lambda n: points.append((n.time, n.mark("Запас"), position(n)))])
    return points


def main():
    net = build()
    with open(RESULTS / "net4_inventory.json", "w", encoding="utf-8") as f:
        json.dump(net.describe(), f, ensure_ascii=False, indent=1)
    print(f"Запаси: попит exp({DEMAND_MEAN}) тижня, перегляд кожні {REVIEW:g} тижні, рівень {LEVEL}, точка "
          f"замовлення {REORDER}, доставка {LEAD:g} тижні, втрата продажу з ймовірністю {LOST_PROBABILITY}\n")
    print("Трасування подій навколо першого замовлення:\n")
    fun_rand.seed(3)
    lines = []
    run(warmup=0.0, time=20.0, trace=10 ** 6, out=lines.append)
    first = next(i for i, s in enumerate(lines) if "Рішення замовити" in s)
    print("\n".join(lines[first - 1:first + 3]))
    delivery = next(i for i, s in enumerate(lines) if "вихід з переходів: Доставка" in s)
    print("\n".join(lines[delivery - 1:delivery + 2]))

    fun_rand.seed(100)
    checked = []
    one = run(hooks=[check_invariant, lambda n: checked.append(1)])
    print(f"Перевірка інваріанта «стан запасу + нестача до рівня = {LEVEL}»: {len(checked)} подій, "
          f"порушень немає")
    print(f"Один прогін: середній запас {one['stock']:.4f}, інтервал між втраченими продажами "
          f"{one['lost_interval']:.4f}")

    runs = replicate(run, SEEDS)
    table = [
        ("середня кількість холодильників у запасі", summary(r["stock"] for r in runs)),
        ("середній час між втраченими продажами, тижні", summary(r["lost_interval"] for r in runs)),
        ("втрачених продажів за тиждень", summary(r["lost_rate"] for r in runs)),
        ("частка втрачених продажів", summary(r["lost_share"] for r in runs)),
        ("частка відкладеного попиту", summary(r["backorder_share"] for r in runs)),
        ("середній невдоволений попит", summary(r["backorders"] for r in runs)),
        ("частка часу без запасу", summary(r["stockout_time"] for r in runs)),
```

```
        ("середній поточний стан запасу", summary(r["position"] for r in runs)),
        ("середній розмір замовлення", summary(r["order_size"] for r in runs)),
        ("середній інтервал між замовленнями, тижні", summary((TIME - WARMUP) / r["orders"] for r in runs)),
    ]
    print(f"\nРЕЗУЛЬТАТИ ({len(SEEDS)} прогонів по {TIME:.0f} тижнів, розгін {WARMUP:.0f} тижнів, "
          f"95 % довірчий інтервал):")
    for label, value in table:
        print(f"  {label:46} {fmt(value)}")
    gaps = [g for r in runs for g in r["lost_gaps"]]
    short = sum(g < REVIEW for g in gaps) / len(gaps)
    print(f"\nІнтервали між втраченими продажами: медіана {statistics.median(gaps):.4f}, коефіцієнт варіації "
          f"{statistics.stdev(gaps) / statistics.fmean(gaps):.4f}, частка інтервалів до 4 тижнів {short:.4f}")

    demand = statistics.fmean(r["demand_rate"] for r in runs)
    sold = statistics.fmean(r["sold_rate"] for r in runs)
    lost = table[2][1][0]
    print(f"\nПеревірка: інтенсивність попиту {demand:.4f} (задано {1 / DEMAND_MEAN:g}), "
          f"продано {sold:.4f} + "
          f"втрачено {lost:.4f} = {sold + lost:.4f}")
    print(f"1 / інтенсивність втрачених продажів = {1 / lost:.4f} тижня")
    direct_runs = replicate(direct, range(1001, 1101))
    ds, dl = summary(r["stock"] for r in direct_runs), summary(r["lost_interval"] for r in direct_runs)
    print(f"Незалежна подієва модель без мережі Петрі: запас {fmt(ds)}, "
          f"інтервал між втраченими продажами {fmt(dl)}")

    fun_rand.seed(7)
    points = trajectory()
    with open(RESULTS / "task4.json", "w", encoding="utf-8") as f:
        json.dump({"table": [{"metric": m, "mean": v[0], "ci": v[1]} for m, v in table],
                   "gaps": {"median": statistics.median(gaps), "short_share": short,
                            "cv": statistics.stdev(gaps) / statistics.fmean(gaps)},
                   "demand_rate": demand, "sold_rate": sold, "direct": {"stock": ds, "lost_interval": dl},
                   "check_events": len(checked), "trajectory": points}, f, ensure_ascii=False, indent=1)


if __name__ == "__main__":
    main()
```

Файл plots.py

```
"""Графіки результатів моделювання (за файлами results/task*.json)."""
import json
from pathlib import Path

import matplotlib

matplotlib.use("Agg")
import matplotlib.pyplot as plt
from matplotlib.ticker import FuncFormatter

RESULTS = Path(__file__).resolve().parent / "results"
BLUE, ORANGE, GRID = "#2a78d6", "#eb6834", "#d9d9d9"


def load(name):
    return json.loads((RESULTS / name).read_text(encoding="utf-8"))


def style(ax, xlabel, ylabel):
    ax.set_xlabel(xlabel)
    ax.set_ylabel(ylabel)
    ax.grid(color=GRID, lw=0.6)
    ax.set_axisbelow(True)
    ax.yaxis.set_major_formatter(FuncFormatter(lambda v, _: f"{v:g}".replace(".", ",")))
    for side in ("top", "right"):
        ax.spines[side].set_visible(False)


def task1():
    hist = load("task1.json")["hist"]
    width, counts = 2 * hist["width"], hist["counts"]
    counts = [sum(counts[i:i + 2]) for i in range(0, len(counts), 2)]
    total = sum(counts)
    limit = int(80 / width)
    fig, ax = plt.subplots(figsize=(7, 3.4))
    ax.bar([i * width for i in range(limit)], [c / total / width for c in counts[:limit]], width=width * 0.9,
           align="edge", color=BLUE)
```

```
    style(ax, "час перебування деталі в системі, хв", "щільність")
    fig.tight_layout()
    fig.savefig(RESULTS / "task1_time.png", dpi=200)
    plt.close(fig)


def task2():
    sweep = load("task2.json")["sweep"]
    fig, axes = plt.subplots(1, 2, figsize=(10, 3.6))
    for variant, color in (("1", BLUE), ("2", ORANGE)):
        xs = [float(a) for a in sweep[variant]]
        rows = list(sweep[variant].values())
        axes[0].errorbar(xs, [r["throughput"][0] for r in rows], yerr=[r["throughput"][1] for r in rows],
                         color=color, marker="o", ms=4, lw=2, capsize=3, label=f"варіант {variant}")
        axes[1].errorbar(xs, [r["time_in_system"][0] for r in rows],
                         yerr=[r["time_in_system"][1] for r in rows],
                         color=color, marker="o", ms=4, lw=2, capsize=3, label=f"варіант {variant}")
    xs = [float(a) for a in sweep["1"]]
    axes[0].plot(xs, [3600 / a for a in xs], color="#888888", ls="--", lw=1,
                 label="інтенсивність надходження")
    style(axes[0], "середній інтервал надходження, с", "деталей за годину")
    style(axes[1], "середній інтервал надходження, с", "час перебування, с")
    axes[1].set_yscale("log")
    for ax in axes:
        ax.legend(frameon=False, fontsize=9)
    fig.tight_layout()
    fig.savefig(RESULTS / "task2_sweep.png", dpi=200)
    plt.close(fig)


def task3():
    sweep = load("task3.json")["sweep"]
    n = [r["n"] for r in sweep]
    fig, axes = plt.subplots(1, 2, figsize=(10, 3.6))
    axes[0].errorbar(n, [r["wait"][0] for r in sweep], yerr=[r["wait"][1] for r in sweep], color=BLUE,
                     marker="o", ms=3.5, lw=2, capsize=2)
    style(axes[0], "кількість місць n", "середнє очікування в черзі, хв")
    axes[1].errorbar(n, [r["revenue"][0] for r in sweep], yerr=[r["revenue"][1] for r in sweep], color=BLUE,
                     marker="o", ms=3.5, lw=2, capsize=2)
    axes[1].axhline(0, color="#888888", lw=1)
    style(axes[1], "кількість місць n", "виручка за день, грн")
    fig.tight_layout()
    fig.savefig(RESULTS / "task3_sweep.png", dpi=200)
    plt.close(fig)


def task4():
    points = load("task4.json")["trajectory"]
    t = [p[0] for p in points]
    fig, ax = plt.subplots(figsize=(10, 3.4))
    ax.step(t, [p[2] for p in points], where="post", color=ORANGE, lw=1.6, label="поточний стан запасу")
    ax.step(t, [p[1] for p in points], where="post", color=BLUE, lw=1.6, label="наявний запас")
    ax.axhline(18, color="#888888", ls="--", lw=1)
    ax.text(0.3, 19.5, "точка замовлення 18", fontsize=9, color="#555555")
    for k in range(4, 61, 4):
        ax.axvline(k, color=GRID, lw=0.6, zorder=0)
    style(ax, "час, тижні", "холодильників")
    ax.grid(False)
    ax.set_ylim(-3, 88)
    ax.legend(frameon=False, fontsize=9, loc="upper center", ncol=2)
    fig.tight_layout()
    fig.savefig(RESULTS / "task4_trajectory.png", dpi=200)
    plt.close(fig)


if __name__ == "__main__":
    task1()
    task2()
    task3()
    task4()
```

3. Результати роботи програми:

Усі чотири моделі побудовано на одному алгоритмі імітації стохастичної часової мережі Петрі (клас `PetriNet`, файл `petri.py`). Мережа складається з позицій з маркерами та переходів, з'єднаних дугами з кратністю. Перехід має часову затримку (стала, експоненційна, рівномірна «середнє ± відхилення» або нормальна), пріоритет і ймовірність спрацювання. Перехід багатоканальний: у ньому одночасно може перебувати кілька маркерів, для кожного запам'ятовується власний момент виходу. Кількість каналів за замовчуванням необмежена, а обмежені ресурси (вільний пристрій, місця на верстаті, роботи) задаються маркерами окремих позицій. Інформаційна дуга (на схемах – штрихова) перевіряє, що у вхідній позиції є потрібна кількість маркерів, але не забирає їх.

Метод `simulate` повторює такі кроки:

- вхід маркерів у переходи: поки є дозволені переходи (у кожній вхідній позиції не менше маркерів, ніж кратність дуги), спрацьовує перехід з найвищим пріоритетом; конфлікт переходів з однаковим пріоритетом розв'язується випадково з ймовірностями, пропорційними параметру probability; маркери забираються з вхідних позицій (крім інформаційних дуг), момент виходу дорівнює поточному часу плюс затримка;
- просування часу до найменшого моменту виходу маркера з переходу з накопиченням статистики;
- вихід з переходів усіх маркерів з цим моментом виходу (одночасні виходи – у порядку входу маркерів у переходи) і поміщення маркерів у вихідні позиції; далі знову вхід маркерів.

Переходи з нульовою затримкою спрацьовують у той самий момент модельного часу, тому через них задаються миттєві рішення (вибір пристрою, посадка, рішення про замовлення). Дозвіл переходу перевіряється лише для переходів, вхідні позиції яких змінилися, що не впливає на результат, але прискорює імітацію.

Статистика позиції – середнє за часом маркування $\Sigma\ M \cdot \Delta t\ /\ T$, мінімум і максимум; статистика переходу – кількість входів і виходів маркерів і середня кількість маркерів у переході (зайнятих каналів). Спостерігачі `watch` накопичують середнє за часом і розподіл довільної функції стану (наприклад, кількості зайнятих пристроїв). Параметр `warmup` задає період розгону, після якого

статистика збирається заново, функції `hooks` викликаються після кожної події (так перевіряються інваріанти), `trace` виводить стан мережі для перших подій.

Звичайні маркери мережі Петрі не розрізняються, тому середній час перебування вимоги в системі можна знайти лише за формулою Літтла $W = L / \lambda$, де L – середня кількість маркерів у позиціях і переходах, що відповідають перебуванню в системі, а λ – інтенсивність виходу. Щоб отримати також розподіл часу, кожен маркер зберігає момент створення, а вихідна дуга з параметром `carry` переносить у вихідну позицію саме ті маркери, що були взяті з указаної вхідної позиції (рух деталі мережею). Спостерігач `on_out` переходу, яким вимога залишає систему, обчислює різницю між поточним часом і моментом створення маркера. Середнє, отримане обома способами, порівнюється для верифікації; від порядку вибору однакових маркерів залежить лише розподіл часу, а середнє за Літтлом – ні.

Метод `describe` зберігає структуру мережі (позиції з початковим маркуванням, переходи із законом і параметрами затримки, пріоритетом, ймовірністю, вхідними, інформаційними і вихідними дугами з кратностями) у файли `results/net*.json`; параметри затримки задано в тій самій формі «середнє, відхилення», що й у бібліотеці PetriObjModelPaint, тому ці ж мережі буде відтворено в наступних практикумах. Для кожної величини виконується серія незалежних прогонів з різними зернами генератора, у таблицях наведено середнє ± півширина 95 % довірчого інтервалу за t-розподілом Стьюдента. На схемах мереж біля переходу вказано затримку (без підпису – нульова) і пріоритет «пр.» (без підпису – 0).

Завдання 1:

Прийнято таке тлумачення умови:

- «постійна швидкість 4 одиниці за 1 хвилину» – детерміновані інтервали між надходженнями 0,25 хв; маркер позиції «Генератор» повертається в неї після кожного спрацювання переходу «Надходження», перша деталь надходить у момент 0,25 хв;
- деталь, що надійшла, одразу опиняється біля П1; рух між сусідніми пристроями триває 1 хв, а від П5 деталь повертається до П1 за 5 хв (повний оберт 4 + 5 = 9 хв);
- біля пристрою деталь не чекає: у той самий момент вона або знімається вільним пристроєм (перехід «Обробка Пі» з пріоритетом 1, який забирає

маркер «Пі вільний» і повертає його після обробки), або рухається далі (перехід руху з пріоритетом 0 і необмеженою кількістю каналів – на стрічці може бути кілька деталей);

- час перебування – від надходження деталі до закінчення її обробки, включно з обробкою;
- оскільки оберт триває рівно 36 інтервалів надходження, деталь, що повернулася, приходить до П1 одночасно з новою; першою обробляється деталь, яка довше перебуває в системі (одночасні виходи впорядковано за моментом входу маркерів у переходи).

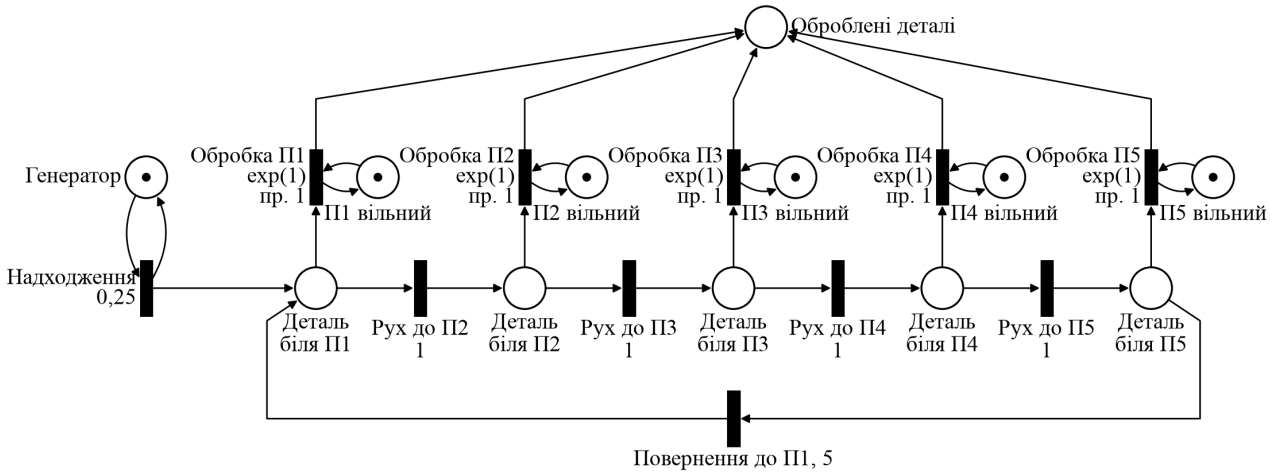


Рисунок 1 – Стохастична мережа Петрі конвеєрної системи

Таблиця 1 – Позиції мережі конвеєрної системи

Позиція	$\mathbf{M_0}$	**Позиція**	$\mathbf{M_0}$
Генератор	1	П3 вільний	1
Деталь біля П1	0	Деталь біля П4	0
П1 вільний	1	П4 вільний	1
Деталь біля П2	0	Деталь біля П5	0
П2 вільний	1	П5 вільний	1
Деталь біля П3	0	Оброблені деталі	0

Таблиця 2 – Переходи мережі конвеєрної системи (час у хвилинах)

Перехід	Затримка	Пр.	Вхідні дуги	Вихідні дуги
Надходження	0,25	0	Генератор	Генератор; Деталь біля П1
Обробка П1	exp(1)	1	Деталь біля П1; П1 вільний	П1 вільний; Оброблені деталі
Рух до П2	1	0	Деталь біля П1	Деталь біля П2
Обробка П2	exp(1)	1	Деталь біля П2; П2 вільний	П2 вільний; Оброблені деталі
Рух до П3	1	0	Деталь біля П2	Деталь біля П3
Обробка П3	exp(1)	1	Деталь біля П3; П3 вільний	П3 вільний; Оброблені деталі
Рух до П4	1	0	Деталь біля П3	Деталь біля П4
Обробка П4	exp(1)	1	Деталь біля П4; П4 вільний	П4 вільний; Оброблені деталі
Рух до П5	1	0	Деталь біля П4	Деталь біля П5
Обробка П5	exp(1)	1	Деталь біля П5; П5 вільний	П5 вільний; Оброблені деталі
Повернення до П1	5	0	Деталь біля П5	Деталь біля П1

Мережу наведено на рис. 1, її позиції та переходи – у табл. 1, 2.

Початок трасування (вхід маркерів у переходи після виходу показано в тому самому блоці):

```
t = 0.2500
  вихід з переходів: Надходження
  вхід у переходи:   Обробка П1, Надходження
  маркування: П2 вільний=1, П3 вільний=1, П4 вільний=1, П5 вільний=1
  у переходах: Надходження=1 (вихід 0.500), Обробка П1=1 (вихід 2.257)

t = 0.5000
  вихід з переходів: Надходження
  вхід у переходи:   Рух до П2, Надходження
  маркування: П2 вільний=1, П3 вільний=1, П4 вільний=1, П5 вільний=1
  у переходах: Надходження=1 (вихід 0.750), Обробка П1=1 (вихід 2.257), Рух до П2=1 (вихід 1.500)

t = 0.7500
  вихід з переходів: Надходження
  вхід у переходи:   Рух до П2, Надходження
  маркування: П2 вільний=1, П3 вільний=1, П4 вільний=1, П5 вільний=1
  у переходах: Надходження=1 (вихід 1.000), Обробка П1=1 (вихід 2.257), Рух до П2=2 (вихід 1.500)

t = 1.0000
  вихід з переходів: Надходження
  вхід у переходи:   Надходження, Рух до П2
  маркування: П2 вільний=1, П3 вільний=1, П4 вільний=1, П5 вільний=1
  у переходах: Надходження=1 (вихід 1.250), Обробка П1=1 (вихід 2.257), Рух до П2=3 (вихід 1.500)

t = 1.2500
  вихід з переходів: Надходження
  вхід у переходи:   Надходження, Рух до П2
  маркування: П2 вільний=1, П3 вільний=1, П4 вільний=1, П5 вільний=1
  у переходах: Надходження=1 (вихід 1.500), Обробка П1=1 (вихід 2.257), Рух до П2=4 (вихід 1.500)

t = 1.5000
  вихід з переходів: Рух до П2, Надходження
  вхід у переходи:   Обробка П2, Рух до П2, Надходження
  маркування: П3 вільний=1, П4 вільний=1, П5 вільний=1
  у переходах: Надходження=1 (вихід 1.750), Обробка П1=1 (вихід 2.257), Рух до П2=4 (вихід
1.750), Обробка П2=1 (вихід 2.300)
```

У момент 0,25 перша деталь забирає маркер «П1 вільний» і починає обробку; наступні деталі, заставши П1 зайнятим, рухаються до П2, і через 1 хв перша з них займає П2.

Моделювання сталого режиму: 20 прогонів по 10000 хв, розгін 1000 хв. Час перебування рахується для деталей, що надійшли між 1000 і 9700 хв, щоб деталі з довгим перебуванням, які не встигли вийти до кінця прогону, не зміщували оцінку.

Таблиця 3 – Результати моделювання конвеєрної системи

Показник	Значення
середній час перебування, хв	9,6205 ± 0,2386
завантаження П1	0,8851 ± 0,0007
завантаження П2	0,8543 ± 0,0010
завантаження П3	0,8132 ± 0,0017
завантаження П4	0,7582 ± 0,0025
завантаження П5	0,6883 ± 0,0035
середня кількість зайнятих пристроїв	3,9991 ± 0,0082
пропускна здатність, деталей/хв	3,9998 ± 0,0007
повернень до П1 на одну деталь	0,7488 ± 0,0257
середня кількість деталей у системі	38,4688 ± 0,9470



Рисунок 2 – Гістограма часу перебування деталі в системі

Таблиця 4 – Характеристики часу перебування деталі, хв

Середнє	Ст. відхилення	Медіана	90 %	95 %	Максимум
9,62	11,71	4,47	22,68	31,55	182,9

Таблиця 5 – Розподіл кількості зайнятих пристроїв (частка часу)

0	1	2	3	4	5
0,0005	0,0085	0,0564	0,2009	0,3932	0,3404

Завантаження пристроїв спадає вздовж конвеєра від 0,885 для П1 до 0,688 для П5 (табл. 3): кожна деталь спочатку проходить повз перші пристрої, тому вони

звільняються найшвидше, а до останніх доходять лише деталі, що застали попередні зайнятими. Частки оброблених деталей розподіляються так само: П1 – 22,0 %, П2 – 21,4 %, П3 – 20,4 %, П4 – 18,9 %, П5 – 17,2 %.

Середня кількість зайнятих пристроїв 3,999; усі п’ять пристроїв зайняті 34,0 % часу (табл. 5), тому значна частина деталей проходить усю стрічку і повертається: у середньому 0,75 повернення на деталь. Середній час перебування 9,62 ± 0,24 хв, але розподіл дуже несиметричний (табл. 4, рис. 2): медіана 4,47 хв, стандартне відхилення 11,7 хв, 5 % деталей перебувають у системі довше 32 хв. Гістограма має піки з періодом 9 хв – кожне повернення додає повний оберт стрічки.

Верифікація моделі:

- в окремому прогоні після кожної з 79954 подій перевірено P-інваріанти: «Пі вільний» + кількість маркерів у переході «Обробка Пі» = 1 для кожного пристрою, «Генератор» + маркер у переході «Надходження» = 1, а кількість деталей, що надійшли, дорівнює сумі оброблених і тих, що перебувають у системі (найбільше 79); порушень немає;
- пропускна здатність 3,9998 деталі за хвилину збігається з інтенсивністю надходження 4 – система стаціонарна, деталі не накопичуються;
- сума завантажень пристроїв (середня кількість зайнятих пристроїв) за законом збереження має дорівнювати $\lambda \cdot E[S] = 4 \cdot 1 = 4$; отримано 3,9991;
- за формулою Літтла $\lambda \cdot W = 3{,}9998 \cdot 9{,}6205 = 38{,}4803$, а середня кількість деталей у системі, обчислена як середнє маркування позицій і переходів мережі, становить 38,4688 – час, виміряний за маркерами з моментом створення, узгоджується з оцінкою, яку дала б мережа зі знеособленими маркерами.

Завдання 2:

Прийнято таке тлумачення умови:

- пункт прибуття, верстати В1, В2 і склад розташовано вздовж однієї лінії переміщення роботів, час ходу між сусідніми точками 6, 7 і 5 с, між несусідніми – сума (наприклад, від складу до пункту прибуття 18 с);
- захоплення і вивільнення деталі триває 8 ± 1 с (рівномірний розподіл на [7; 9]) і виконується в точці, де лежить деталь; обробка на верстаті починається після вивільнення деталі роботом;

- робота призначають на переміщення лише тоді, коли на верстаті призначення є вільне місце, і це місце резервується в момент призначення – інакше робот міг би привезти деталь до заповненого верстата і чекати з нею; місце звільняється, коли наступний робот захопить оброблену деталь;
- час обробки N(60; 10) обмежено знизу нулем (від'ємне значення практично неможливе); черга на пункті прибуття і склад не обмежені; можливі зіткнення роботів на лінії не враховуються;
- варіант 1: робот k закріплений за шляхом k і після вивільнення деталі одразу холостим ходом повертається на початок свого шляху (6, 7 або 5 с) – так наступна деталь не чекає на холостий хід;
- варіант 2: робот після вивільнення залишається в точці, куди привіз деталь (позиції «Вільний робот: вхід / B1 / B2 / склад»). Для кожного шляху є чотири переходи «Виклик робота: L → шлях» із затримкою, що дорівнює часу холостого ходу від точки L до початку шляху, і пріоритетом 100 − (час ходу), тому з кількох вільних роботів спрацьовує перехід найближчого; якщо кілька деталей чекають, першою обслуговується та, до якої найближчий вільний робот ближче;
- на початку роботи всі роботи вільні й стоять на початках шляхів (пункт прибуття, B1, B2);
- пропускна здатність – кількість деталей, доставлених на склад за годину.

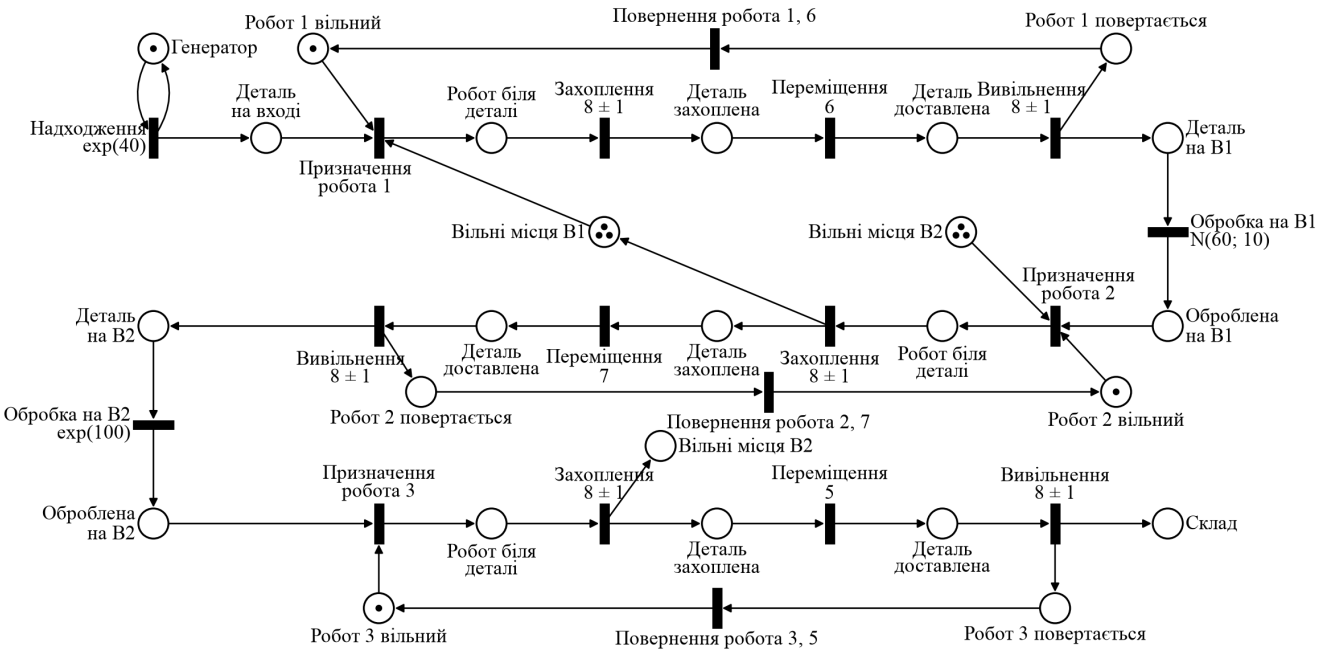


Рисунок 3 – Мережа Петрі гнучкої виробничої системи, варіант 1 (позицію «Вільні місця B2» зображено двічі)

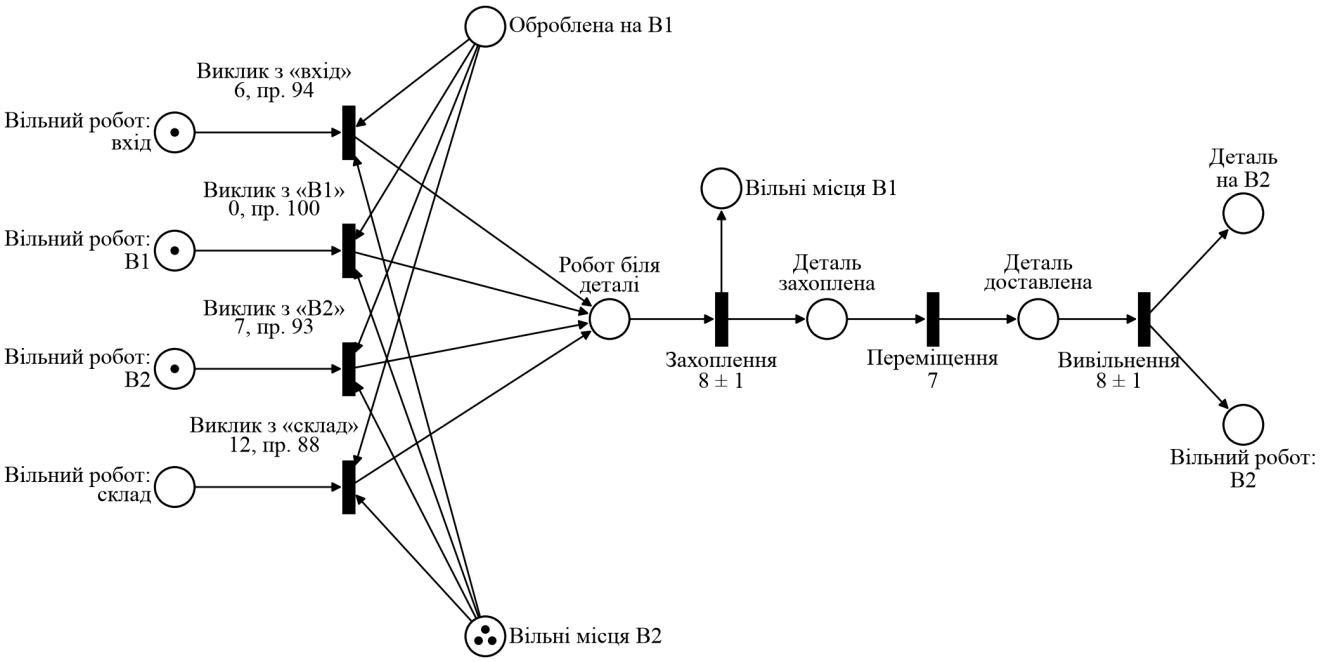


Рисунок 4 – Варіант 2: виклик найближчого робота для шляху B1–B2 (позицію «Вільний робот: B2» зображено двічі)

Мережу варіанта 1 наведено на рис. 3: деталь проходить три однакові ланцюги «призначення робота – захоплення – переміщення – вивільнення», між якими стоять переходи обробки на верстатах з необмеженою кількістю каналів (кількість деталей на верстаті обмежують три маркери позиції «Вільні місця»). На

схемі місця, позначені «Робот біля деталі», «Деталь захоплена», «Деталь доставлена», у мережі мають суфікс шляху, наприклад «Деталь захоплена (B1–B2)». У варіанті 2 (рис. 4) переходи «Призначення робота k» і ланцюги повернення роботів замінено підмережею виклику: для кожного шляху чотири переходи виклику (на рисунку – для шляху B1–B2), а вивільнення деталі повертає маркер робота в позицію точки призначення. Повний перелік позицій і переходів наведено в табл. 6–8.

Таблиця 6 – Позиції мереж ГВС (варіант 1 і варіант 2)

Позиція	**M_0**	**Позиція**	**M_0**
Робот 1 вільний	1	Склад	0
Робот 1 повертається	0	Робот біля деталі (вхід–B1)	0
Робот 2 вільний	1	Деталь захоплена (вхід–B1)	0
Робот 2 повертається	0	Деталь доставлена (вхід–B1)	0
Робот 3 вільний	1	Робот біля деталі (B1–B2)	0
Робот 3 повертається	0	Деталь захоплена (B1–B2)	0
Генератор	1	Деталь доставлена (B1–B2)	0
Деталь на вході	0	Робот біля деталі (B2–склад)	0
Вільні місця B1	3	Деталь захоплена (B2–склад)	0
Деталь на B1	0	Деталь доставлена (B2–склад)	0
Оброблена на B1	0	Вільний робот: вхід	1
Вільні місця B2	3	Вільний робот: B1	1
Деталь на B2	0	Вільний робот: B2	1
Оброблена на B2	0	Вільний робот: склад	0

Таблиця 7 – Переходи мережі ГВС, варіант 1 (час у секундах)

Перехід	Затримка	Пр.	Вхідні дуги	Вихідні дуги
Надходження	exp(40)	0	Генератор	Генератор; Деталь на вході
Обробка на B1	N(60; 10)	0	Деталь на B1	Оброблена на B1
Обробка на B2	exp(100)	0	Деталь на B2	Оброблена на B2
Призначення робота 1	0	0	Деталь на вході; Вільні місця B1; Робот 1 вільний	Робот біля деталі (вхід–B1)
Захоплення (вхід–B1)	8 ± 1	0	Робот біля деталі (вхід–B1)	Деталь захоплена (вхід–B1)
Переміщення (вхід–B1)	6	0	Деталь захоплена (вхід–B1)	Деталь доставлена (вхід–B1)
Вивільнення (вхід–B1)	8 ± 1	0	Деталь доставлена (вхід–B1)	Деталь на B1; Робот 1 повертається
Повернення робота 1	6	0	Робот 1 повертається	Робот 1 вільний
Призначення робота 2	0	0	Оброблена на B1; Вільні місця B2; Робот 2 вільний	Робот біля деталі (B1–B2)
Захоплення (B1–B2)	8 ± 1	0	Робот біля деталі (B1–B2)	Деталь захоплена (B1–B2); Вільні місця B1
Переміщення (B1–B2)	7	0	Деталь захоплена (B1–B2)	Деталь доставлена (B1–B2)
Вивільнення (B1–B2)	8 ± 1	0	Деталь доставлена (B1–B2)	Деталь на B2; Робот 2 повертається
Повернення робота 2	7	0	Робот 2 повертається	Робот 2 вільний
Призначення робота 3	0	0	Оброблена на B2; Робот 3 вільний	Робот біля деталі (B2–склад)
Захоплення (B2–склад)	8 ± 1	0	Робот біля деталі (B2–склад)	Деталь захоплена (B2–склад); Вільні місця B2
Переміщення (B2–склад)	5	0	Деталь захоплена (B2–склад)	Деталь доставлена (B2–склад)
Вивільнення (B2–склад)	8 ± 1	0	Деталь доставлена (B2–склад)	Склад; Робот 3 повертається
Повернення робота 3	5	0	Робот 3 повертається	Робот 3 вільний

Таблиця 8 – Переходи виклику роботів, варіант 2

Перехід	Затримка	Пр.	Вхідні дуги	Вихідні дуги
Виклик робота: вхід → вхід–B1	0	100	Деталь на вході; Вільні місця B1; Вільний робот: вхід	Робот біля деталі (вхід–B1)
Виклик робота: B1 → вхід–B1	6	94	Деталь на вході; Вільні місця B1; Вільний робот: B1	Робот біля деталі (вхід–B1)
Виклик робота: B2 → вхід–B1	13	87	Деталь на вході; Вільні місця B1; Вільний робот: B2	Робот біля деталі (вхід–B1)
Виклик робота: склад → вхід–B1	18	82	Деталь на вході; Вільні місця B1; Вільний робот: склад	Робот біля деталі (вхід–B1)
Виклик робота: вхід → B1–B2	6	94	Оброблена на B1; Вільні місця B2; Вільний робот: вхід	Робот біля деталі (B1–B2)
Виклик робота: B1 → B1–B2	0	100	Оброблена на B1; Вільні місця B2; Вільний робот: B1	Робот біля деталі (B1–B2)
Виклик робота: B2 → B1–B2	7	93	Оброблена на B1; Вільні місця B2; Вільний робот: B2	Робот біля деталі (B1–B2)
Виклик робота: склад → B1–B2	12	88	Оброблена на B1; Вільні місця B2; Вільний робот: склад	Робот біля деталі (B1–B2)
Виклик робота: вхід → B2–склад	13	87	Оброблена на B2; Вільний робот: вхід	Робот біля деталі (B2–склад)
Виклик робота: B1 → B2–склад	7	93	Оброблена на B2; Вільний робот: B1	Робот біля деталі (B2–склад)
Виклик робота: B2 → B2–склад	0	100	Оброблена на B2; Вільний робот: B2	Робот біля деталі (B2–склад)
Виклик робота: склад → B2–склад	5	95	Оброблена на B2; Вільний робот: склад	Робот біля деталі (B2–склад)

Позиції «Робот k вільний» і «Робот k повертається» є лише у варіанті 1, позиції «Вільний робот: …» – лише у варіанті 2; решта переходів варіанта 2 збігається з табл. 7, крім того, що вивільнення ставить маркер у позицію «Вільний робот» точки призначення. Початок трасування варіанта 1 і фрагмент варіанта 2, у якому для деталі на пункті прибуття викликано робота з В1 (6 с ходу), а не з В2 (13 с):

```
t = 57.7488
  вихід з переходів: Надходження
  вхід у переходи:   Надходження, Призначення робота 1
  маркування: Вільні місця В1=2, Вільні місця В2=3, Робот 2 вільний=1, Робот 3 вільний=1
  у переходах: Надходження=1 (вихід 94.797), Призначення робота 1=1 (вихід 57.749)

t = 57.7488
  вихід з переходів: Призначення робота 1
  вхід у переходи:   Захоплення (вхід-В1)
  маркування: Вільні місця В1=2, Вільні місця В2=3, Робот 2 вільний=1, Робот 3 вільний=1
  у переходах: Надходження=1 (вихід 94.797), Захоплення (вхід-В1)=1 (вихід 65.059)

t = 65.0587
  вихід з переходів: Захоплення (вхід-В1)
  вхід у переходи:   Переміщення (вхід-В1)
  маркування: Вільні місця В1=2, Вільні місця В2=3, Робот 2 вільний=1, Робот 3 вільний=1
  у переходах: Надходження=1 (вихід 94.797), Переміщення (вхід-В1)=1 (вихід 71.059)

t = 71.0587
  вихід з переходів: Переміщення (вхід-В1)
  вхід у переходи:   Вивільнення (вхід-В1)
  маркування: Вільні місця В1=2, Вільні місця В2=3, Робот 2 вільний=1, Робот 3 вільний=1
  у переходах: Надходження=1 (вихід 94.797), Вивільнення (вхід-В1)=1 (вихід 78.192)

t = 148.6054
  вихід з переходів: Надходження
  вхід у переходи:   Виклик робота: В1 → вхід-В1, Надходження
  маркування: Вільні місця В1=1, Вільні місця В2=2, Вільний робот: В2=1
  у переходах: Надходження=1 (вихід 157.509), Виклик робота: В1 → вхід-В1=1 (вихід 154.605),
Захоплення (В1-В2)=1 (вихід 156.948)
```

Таблиця 9 – Результати моделювання ГВС (надходження кожні 40 с)

Показник	**Варіант 1**	**Варіант 2**
інтенсивність надходження, деталей/год	89,43 ± 1,13	89,98 ± 0,94
пропускна здатність, деталей/год	76,08 ± 0,50	71,79 ± 0,50
середній час перебування деталі, с	9282,23 ± 629,86	11839,97 ± 663,42
середня черга на вході	219,81 ± 16,54	282,66 ± 16,52
середня кількість зайнятих роботів	1,78 ± 0,01	1,71 ± 0,01
зайняті місця В1 (з резервом)	3,00 ± 0,00	2,95 ± 0,00
зайняті місця В2 (з резервом)	2,90 ± 0,00	2,79 ± 0,00
деталей в обробці на В1	1,27 ± 0,01	1,20 ± 0,01
деталей в обробці на В2	2,10 ± 0,01	2,02 ± 0,01
оброблені на В1 чекають робота	1,10 ± 0,01	0,81 ± 0,02
оброблені на В2 чекають робота	0,14 ± 0,00	0,02 ± 0,00
роботів у холостому ході	0,38 ± 0,00	0,39 ± 0,00

Таблиця 10 – Необмежене надходження деталей

Показник	Варіант 1	Варіант 2
пропускна здатність, деталей/год	75,95 ± 0,38	72,13 ± 0,45
середня кількість зайнятих роботів	1,77 ± 0,01	1,71 ± 0,01
зайняті місця B1 (з резервом)	3,00 ± 0,00	2,95 ± 0,00
зайняті місця B2 (з резервом)	2,90 ± 0,00	2,79 ± 0,01
деталей в обробці на B1	1,26 ± 0,01	1,20 ± 0,01
деталей в обробці на B2	2,11 ± 0,01	2,01 ± 0,01
оброблені на B1 чекають робота	1,10 ± 0,01	0,80 ± 0,01
оброблені на B2 чекають робота	0,14 ± 0,00	0,02 ± 0,00
роботів у холостому ході	0,38 ± 0,00	0,39 ± 0,00

Таблиця 11 – Залежність від середнього інтервалу надходження

Інтервал, с	Вар. 1, дет./год	Вар. 2, дет./год	Вар. 1: час, с	Вар. 2: час, с
45	75,62 ± 0,54	72,24 ± 0,40	3551 ± 685	6465 ± 663
50	72,30 ± 0,81	71,06 ± 0,53	880 ± 212	1736 ± 449
55	65,47 ± 0,68	65,14 ± 0,81	425 ± 35	493 ± 24
60	59,73 ± 1,02	59,89 ± 0,77	340 ± 14	382 ± 18
70	51,41 ± 0,50	50,98 ± 0,52	285 ± 5	309 ± 10
80	45,16 ± 0,59	45,41 ± 0,60	264 ± 4	280 ± 3

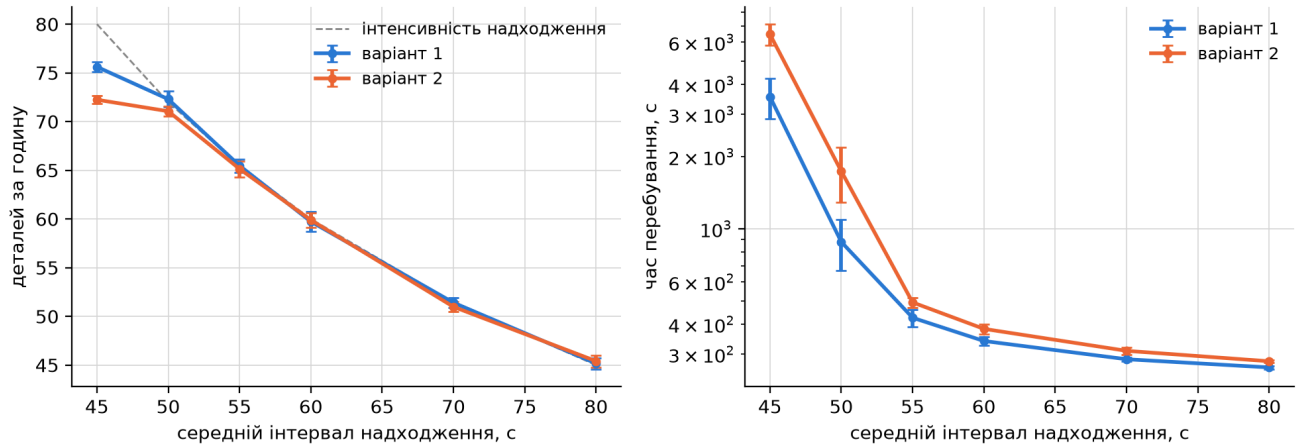


Рисунок 5 – Пропускна здатність і час перебування деталі залежно від інтервалу надходження

Моделювання: 20 прогонів по 100000 с (≈ 28 год) з розгоном 10000 с для кожного режиму. За заданого інтервалу 40 с (≈ 90 деталей за годину) жоден варіант не встигає обробляти потік (табл. 9): на склад надходить 76,1 деталі за годину у варіанті 1 і 71,8 у варіанті 2, черга на пункті прибуття постійно зростає, а середній час перебування залежить від тривалості прогону і характеризує лише накопичення черги. Тому пропускну здатність визначено також при необмеженому

надходженні (на вході 100000 деталей, табл. 10): 75,9 і 72,1 деталі за годину – на 5,3 % більше у варіанті 1. Значення збігаються з результатами при інтервалі 40 с, тобто в цьому режимі система працює на межі своїх можливостей.

Вузьке місце – верстат B2. Його місце зайняте не лише на час обробки (100 с), а й на захоплення деталі на B1, переміщення і вивільнення (≈ 23 с), очікування робота та захоплення (≈ 8 с), тобто ≈ 131 с і більше, тому три місця дають не більше ≈ 82 деталей за годину. У середньому лише 2,11 з 2,90 зайнятих місць B2 відведено під обробку. Верстат B1 при цьому заблокований: усі три місця зайняті, а 1,10 оброблених деталі в середньому чекають вільного місця на B2.

Варіант 1 кращий, бо закріплений робот одразу після вивільнення повертається на початок свого шляху, паралельно з обробкою: робот 2 чекає на B1, робот 3 – на B2, і як тільки звільняється місце на B2, деталь з B1 захоплюється без холостого ходу. У варіанті 2 робот залишається там, куди привіз деталь, зокрема на складі – звідти виконується 33,3 % викликів. Середній холостий хід до деталі становить 12,9 с для шляху вхід–B1, 4,4 с для B1–B2 і 2,3 с для B2–склад, причому під час холостого ходу до деталі на B1 місце на B2 вже зарезервоване, тобто холостий хід подовжує зайнятість вузького місця. Вибір найближчого робота мінімізує лише поточний холостий хід і не враховує, де робот знадобиться далі.

За меншого навантаження (табл. 11, рис. 5) обидва варіанти встигають обробляти потік, і пропускна здатність дорівнює інтенсивності надходження, але час перебування деталі у варіанті 1 менший при кожному інтервалі (наприклад, при 60 с – 340 проти 382 с). При інтервалі 50 с варіант 2 вже перевантажений (71 деталь за годину при 72 тих, що надходять), а варіант 1 ще ні. Отже, з точки зору пропускної здатності кращим є закріплення по одному роботу за кожним шляхом.

Верифікація моделі:

- у кожному варіанті після кожної події перевірено P-інваріанти: кожен робот перебуває рівно в одному стані (вільний, у ланцюгу переміщення, повертається або в холостому ході), а для кожного верстата вільні + зарезервовані + зайняті деталлю місця = 3; порушень немає;
- інтенсивність надходження в прогонах 89,4 деталі за годину відповідає заданій 3600 / 40 = 90;
- за законом збереження середня кількість деталей в обробці дорівнює добутку пропускної здатності на середній час обробки: для B1 X·60 = 1,266 (імітація 1,265), для B2 X·100 = 2,110 (імітація 2,109);

- у варіанті 1 кожен робот на одну деталь витрачає 8 + t + 8 + t секунд (t – час ходу шляху), тобто 28, 30 і 26 с, тому зайнятих роботів має бути X·84 = 1,772; імітація дає 1,772.

Завдання 3:

Прийнято таке тлумачення умови:

- маршрут має дві кінцеві зупинки З1 і З2; мікроавтобус, висадивши пасажирів, «їде у зворотному напрямку тим же чином», тобто так само набирає пасажирів на іншій кінцевій; на кожну зупинку пасажири підходять незалежно з інтервалом 0,5 ± 0,2 хв (рівномірний розподіл);
- посадка миттєва; якщо на зупинці є мікроавтобус з вільними місцями, пасажир сідає в нього, причому в В – лише коли на зупинці немає А (переходи посадки в А мають вищий пріоритет, а А, заповнившись, одразу відправляється); інакше пасажир стає в чергу;
- у черзі може бути не більше 30 осіб (позиція «Місця в черзі» з 30 маркерами); пасажир, що застав 30 осіб у черзі, йде на інший маршрут;
- мікроавтобус відправляється в момент, коли в ньому n пасажирів (дуга кратності n), рейс триває 20 ± 5 хв для А і 30 ± 5 хв для В, висадка – 5 ± 1 хв на кінцевій, після чого починається посадка там же;
- на початку дня черги порожні, А стоїть на З1, В – на З2, обидва порожні; моделюється день тривалістю 10 год = 600 хв (кінцева імітація без розгону), по 50 днів для кожного n;
- час очікування в черзі – від приходу пасажира на зупинку до посадки для всіх пасажирів, що сіли в мікроавтобус (для тих, хто сів одразу, він дорівнює нулю); окремо наведено середнє для тих, хто стояв у черзі; час, проведений у мікроавтобусі до відправлення, очікуванням у черзі не вважається;
- виручка за день = 20 грн · (кількість пасажирів, що сіли) − 20 грн · (кількість пасажирів, що пішли на інший маршрут); плата береться при посадці.

Таблиця 12 – Позиції мережі маршруту мікроавтобусів

Позиція	M_0	Позиція	M_0
Генератор 31	1	А на 32	0
Пасажир на 31	0	Вільні місця А на 32	0
Черга на 31	0	Пасажири в А на 32	0
Місця в черзі 31	30	А прибув на 32	0
Пішли з 31	0	В на 31	0
Генератор 32	1	Вільні місця В на 31	0
Пасажир на 32	0	Пасажири в В на 31	0
Черга на 32	0	В прибув на 31	0
Місця в черзі 32	30	В на 32	1
Пішли з 32	0	Вільні місця В на 32	n
А на 31	1	Пасажири в В на 32	0
Вільні місця А на 31	n	В прибув на 32	0
Пасажири в А на 31	0	Перевезені пасажири	0
А прибув на 31	0		

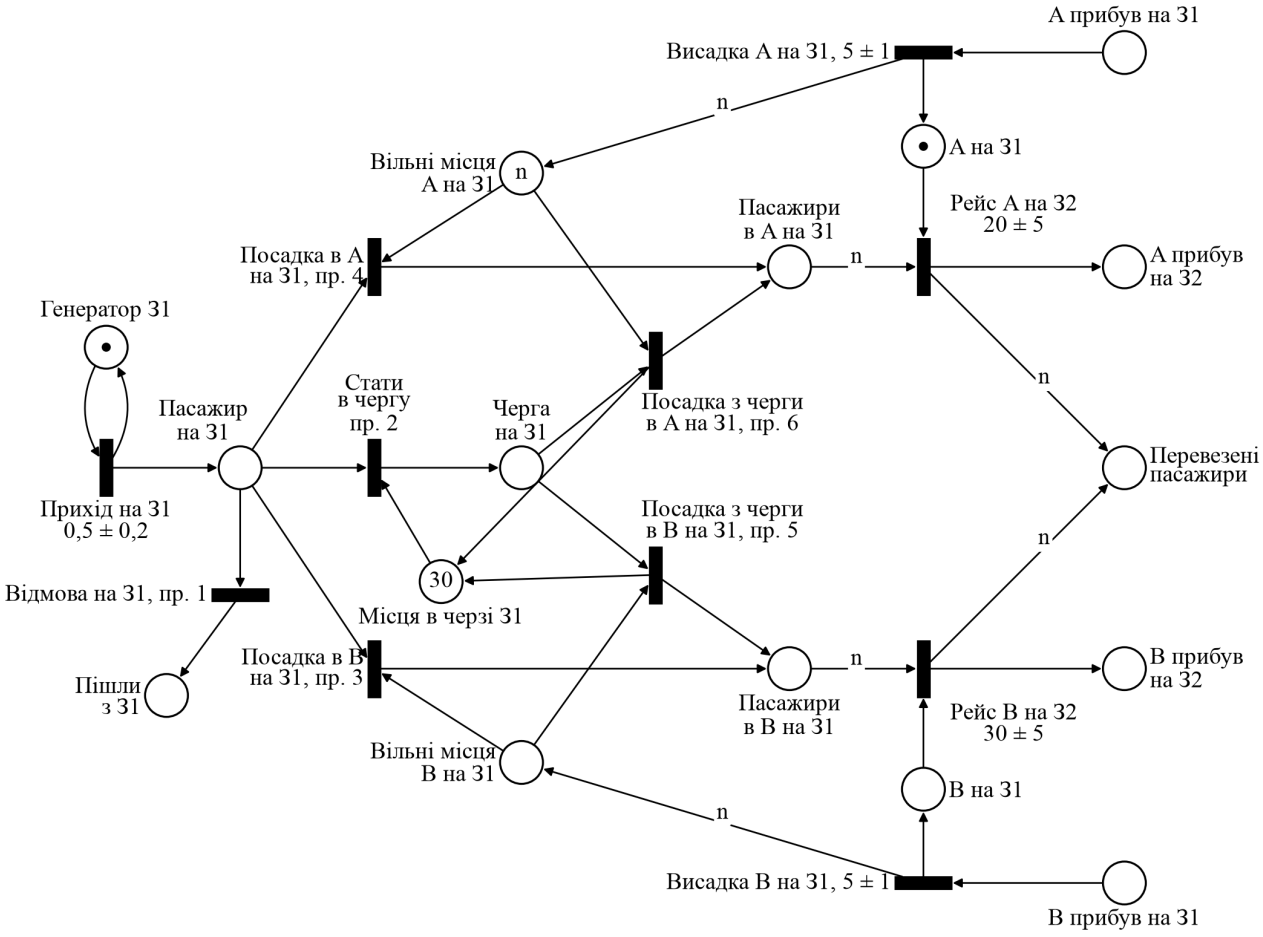


Рисунок 6 – Мережа Петрі маршруту: підмережа зупинки 31 (підмережа 32 симетрична)

Таблиця 13 – Переходи мережі маршруту мікроавтобусів (час у хвилинах)

Перехід	Затримка	Пр.	Вхідні дуги	Вихідні дуги
Прихід на З1	0,5 ± 0,2	0	Генератор З1	Генератор З1; Пасажир на З1
Посадка з черги в А на З1	0	6	Черга на З1; Вільні місця А на З1	Пасажири в А на З1; Місця в черзі З1
Посадка в А на З1	0	4	Пасажир на З1; Вільні місця А на З1	Пасажири в А на З1
Посадка з черги в В на З1	0	5	Черга на З1; Вільні місця В на З1	Пасажири в В на З1; Місця в черзі З1
Посадка в В на З1	0	3	Пасажир на З1; Вільні місця В на З1	Пасажири в В на З1
Стати в чергу на З1	0	2	Пасажир на З1; Місця в черзі З1	Черга на З1
Відмова на З1	0	1	Пасажир на З1	Пішли з З1
Прихід на З2	0,5 ± 0,2	0	Генератор З2	Генератор З2; Пасажир на З2
Посадка з черги в А на З2	0	6	Черга на З2; Вільні місця А на З2	Пасажири в А на З2; Місця в черзі З2
Посадка в А на З2	0	4	Пасажир на З2; Вільні місця А на З2	Пасажири в А на З2
Посадка з черги в В на З2	0	5	Черга на З2; Вільні місця В на З2	Пасажири в В на З2; Місця в черзі З2
Посадка в В на З2	0	3	Пасажир на З2; Вільні місця В на З2	Пасажири в В на З2
Стати в чергу на З2	0	2	Пасажир на З2; Місця в черзі З2	Черга на З2
Відмова на З2	0	1	Пасажир на З2	Пішли з З2
Рейс А З1→З2	20 ± 5	0	А на З1; Пасажири в А на З1 ×n	А прибув на З2; Перевезені пасажири ×n
Висадка А на З2	5 ± 1	0	А прибув на З2	А на З2; Вільні місця А на З2 ×n
Рейс А З2→З1	20 ± 5	0	А на З2; Пасажири в А на З2 ×n	А прибув на З1; Перевезені пасажири ×n
Висадка А на З1	5 ± 1	0	А прибув на З1	А на З1; Вільні місця А на З1 ×n
Рейс В З1→З2	30 ± 5	0	В на З1; Пасажири в В на З1 ×n	В прибув на З2; Перевезені пасажири ×n
Висадка В на З2	5 ± 1	0	В прибув на З2	В на З2; Вільні місця В на З2 ×n
Рейс В З2→З1	30 ± 5	0	В на З2; Пасажири в В на З2 ×n	В прибув на З1; Перевезені пасажири ×n
Висадка В на З1	5 ± 1	0	В прибув на З1	В на З1; Вільні місця В на З1 ×n

Мережу для зупинки З1 наведено на рис. 6; для З2 вона симетрична, підмережі з’єднуються переходами рейсів. Позиції «А на З1», «Вільні місця А на З1», «Пасажири в А на З1» описують мікроавтобус, що стоїть на посадці; сума вільних місць і пасажирів у ньому дорівнює n. Повний перелік позицій і переходів – у табл. 12, 13. Початок трасування для n = 3:

```
t = 0.6055
  вихід з переходів: Прихід на З2
  вхід у переходи:   Посадка в В на З2, Прихід на З2
  маркування: Місця в черзі З1=30, Місця в черзі З2=30, А на З1=1, Вільні місця А на З1=3, В на
З2=1, Вільні місця В на З2=2
  у переходах: Прихід на З1=1 (вихід 0.639), Прихід на З2=1 (вихід 1.008), Посадка в В на З2=1
(вихід 0.606)

t = 0.6055
  вихід з переходів: Посадка в В на З2
  маркування: Місця в черзі З1=30, Місця в черзі З2=30, А на З1=1, Вільні місця А на З1=3, В на
З2=1, Вільні місця В на З2=2, Пасажири в В на З2=1
  у переходах: Прихід на З1=1 (вихід 0.639), Прихід на З2=1 (вихід 1.008)

...

t = 1.6978
  вихід з переходів: Посадка в А на З1
  вхід у переходи:   Рейс А З1→З2
  маркування: Місця в черзі З1=30, Місця в черзі З2=30
  у переходах: Прихід на З1=1 (вихід 2.009), Прихід на З2=1 (вихід 2.103), Рейс А З1→З2=1 (вихід
25.055), Рейс В З2→З1=1 (вихід 27.426)

t = 2.0091
  вихід з переходів: Прихід на З1
  вхід у переходи:   Стати в чергу на З1, Прихід на З1
  маркування: Місця в черзі З1=29, Місця в черзі З2=30
  у переходах: Прихід на З1=1 (вихід 2.482), Стати в чергу на З1=1 (вихід 2.009), Прихід на З2=1
(вихід 2.103), Рейс А З1→З2=1 (вихід 25.055), Рейс В З2→З1=1 (вихід 27.426)
```

Пасажири, що підходять до зупинок, одразу сідають у мікроавтобуси, які там стоять (А на З1, В на З2). Коли в А стало три пасажири (кратність дуги n = 3), перехід «Рейс А З1→З2» забирає їх і маркер «А на З1»; наступний пасажир на З1 вже не має вільних місць і стає в чергу.

Таблиця 14 – Результати моделювання маршруту за кількістю місць n

n	Очікування, хв	Серед тих, хто чекав, хв	Сіли	Пішли	Виручка, грн
1	287,69 ± 1,01	302,01 ± 1,05	42,2	2296,2	−45080 ± 68
2	265,60 ± 0,89	278,87 ± 0,95	84,1	2254,7	−43411 ± 61
3	210,61 ± 0,82	221,11 ± 0,89	126,4	2211,6	−41703 ± 72
4	170,40 ± 0,78	178,91 ± 0,86	168,2	2170,4	−40043 ± 71
5	142,85 ± 0,51	150,00 ± 0,56	209,9	2129,5	−38392 ± 72
6	121,63 ± 0,47	127,71 ± 0,53	252,2	2086,4	−36684 ± 72
7	105,82 ± 0,43	111,14 ± 0,47	293,0	2045,9	−35058 ± 96
8	93,47 ± 0,48	98,15 ± 0,53	335,8	2003,8	−33359 ± 99
9	83,74 ± 0,38	87,97 ± 0,42	375,1	1964,4	−31786 ± 97
10	75,55 ± 0,31	79,36 ± 0,34	417,4	1923,1	−30113 ± 106
11	68,81 ± 0,29	72,27 ± 0,31	458,7	1882,6	−28479 ± 91
12	62,85 ± 0,27	66,01 ± 0,29	501,6	1840,7	−26781 ± 111
13	58,07 ± 0,25	61,02 ± 0,27	539,2	1802,9	−25272 ± 111
14	53,55 ± 0,20	56,25 ± 0,22	583,2	1759,7	−23528 ± 135
15	49,66 ± 0,21	52,17 ± 0,23	624,3	1716,9	−21853 ± 130
16	46,12 ± 0,20	48,47 ± 0,22	665,3	1679,2	−20278 ± 115
17	43,09 ± 0,22	45,47 ± 0,25	704,5	1640,4	−18719 ± 156
18	40,23 ± 0,23	42,64 ± 0,24	746,2	1600,3	−17081 ± 163
19	37,67 ± 0,21	40,25 ± 0,25	783,6	1561,1	−15550 ± 172
20	35,40 ± 0,21	38,16 ± 0,24	824,8	1522,6	−13957 ± 176
21	33,39 ± 0,18	36,43 ± 0,25	859,6	1487,8	−12564 ± 189
22	31,41 ± 0,19	34,77 ± 0,22	895,8	1451,7	−11118 ± 147
23	29,76 ± 0,17	33,40 ± 0,29	929,8	1419,7	−9799 ± 198
24	28,28 ± 0,17	32,05 ± 0,29	964,3	1385,8	−8430 ± 203
25	26,87 ± 0,18	31,33 ± 0,40	990,9	1359,7	−7374 ± 257

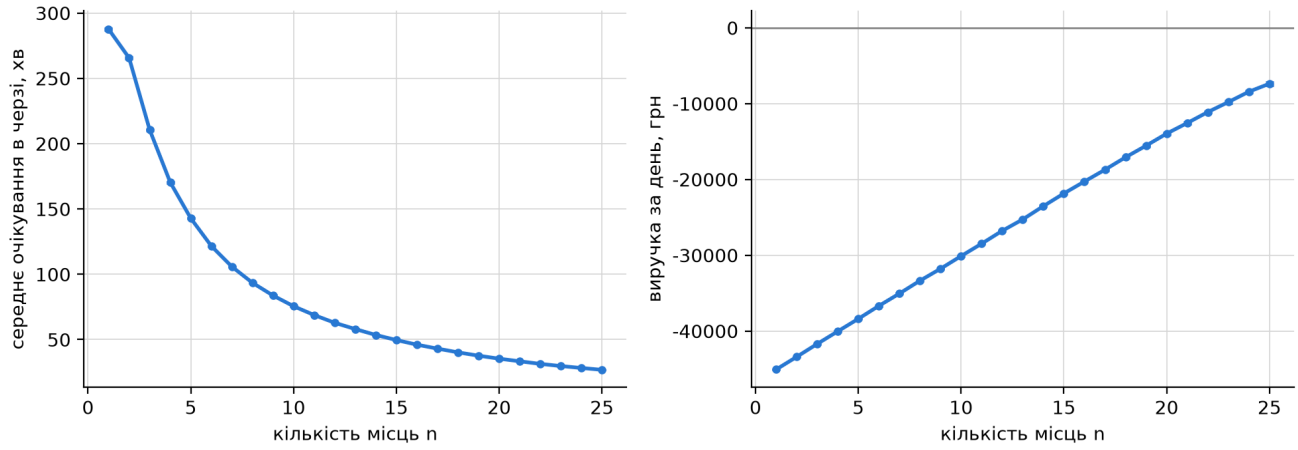


Рисунок 7 – Середній час очікування в черзі та виручка за день залежно від n

Середній час очікування монотонно зменшується зі збільшенням n (табл. 14, рис. 7), і мінімальний він при найбільшому допустимому n = 25: 26,87 ± 0,18 хв для всіх пасажирів, що сіли, і 31,33 ± 0,40 хв для тих, хто стояв у черзі. Виручка за день при n = 25 становить −7374 ± 257 грн: дохід 19819 грн від 991 пасажирів і втрати 27193 грн від 1360 пасажирів, що пішли.

Причина – маршрут сильно перевантажений. На кожну зупинку за день приходить ≈ 1200 пасажирів (2 за хвилину), а два мікроавтобуси, навіть якщо щоразу заповнюються одразу з черги, перевозять в одному напрямку $n\cdot(1/50 + 1/70)$ пасажирів за хвилину (А робить коло $2\cdot(20 + 5) = 50$ хв, В – 70 хв), тобто 0,857 за $n = 25$. Черга майже весь час заповнена, понад половина пасажирів іде на інший маршрут, і час очікування визначається тим, як швидко мікроавтобуси розвозять 30 осіб черги, – приблизно обернено пропорційно n. Тому за будь-якого $n \le 25$ виручка від'ємна; вона зростає з n приблизно на 1570 грн на кожне місце. Щоб перевезти весь потік, потрібно було б $n \approx 2 / (1/50 + 1/70) \approx 58$ місць або більше мікроавтобусів.

Верифікація моделі:

- для $n = 10$ в 2467 стійких станах (коли в переходах немає маркерів з нульовою затримкою) перевірено інваріанти: кожен мікроавтобус перебуває рівно в одному стані (на посадці, у рейсі, на висадці); вільні місця + пасажири в мікроавтобусі = n, якщо він на зупинці, і 0 інакше; черга + вільні місця в черзі = 30; черги немає, коли на зупинці стоїть мікроавтобус; кількість пасажирів, що прийшли, дорівнює сумі тих, що сіли, пішли і стоять у черзі; порушень немає;
- для довгого прогону (60000 хв, $n = 25$) за формулою Літтла середня черга на зупинці $\lambda\cdot W = 0{,}7401 \cdot 31{,}9676 = 23{,}6598$ збігається з виміряною 23,6522;
- у тому ж прогоні в одному напрямку перевезено 0,822 пасажира за хвилину при оцінці зверху 0,857; рейсів А 2286, В 1660 при оцінках 2400 і 1714; невелика різниця пояснюється тим, що черга не завжди містить n осіб, коли під'їжджає мікроавтобус, і він чекає заповнення.

Завдання 4:

Прийнято таке тлумачення умови:

- одиниця часу – тиждень; кожен покупець купує один холодильник, інтервали між покупцями – exp(0,2), тобто в середньому 5 покупців за тиждень;
- якщо холодильника немає, з ймовірністю 0,8 продаж втрачено, з ймовірністю 0,2 покупець оформлює повторне замовлення (невдоволений попит) і отримує холодильник першим, коли надійде партія;

- запас переглядається в моменти 4, 8, 12, … тижнів; поточний стан запасу = наявний запас + замовлені холодильники − невдоволений попит; якщо він не більший за 18, замовляється 72 − (поточний стан) холодильників, партія надходить через 3 тижні (одночасно в дорозі не більше однієї партії, бо 3 < 4);
- середній час між продажами, що не відбулися, – середній інтервал між послідовними втраченими продажами.

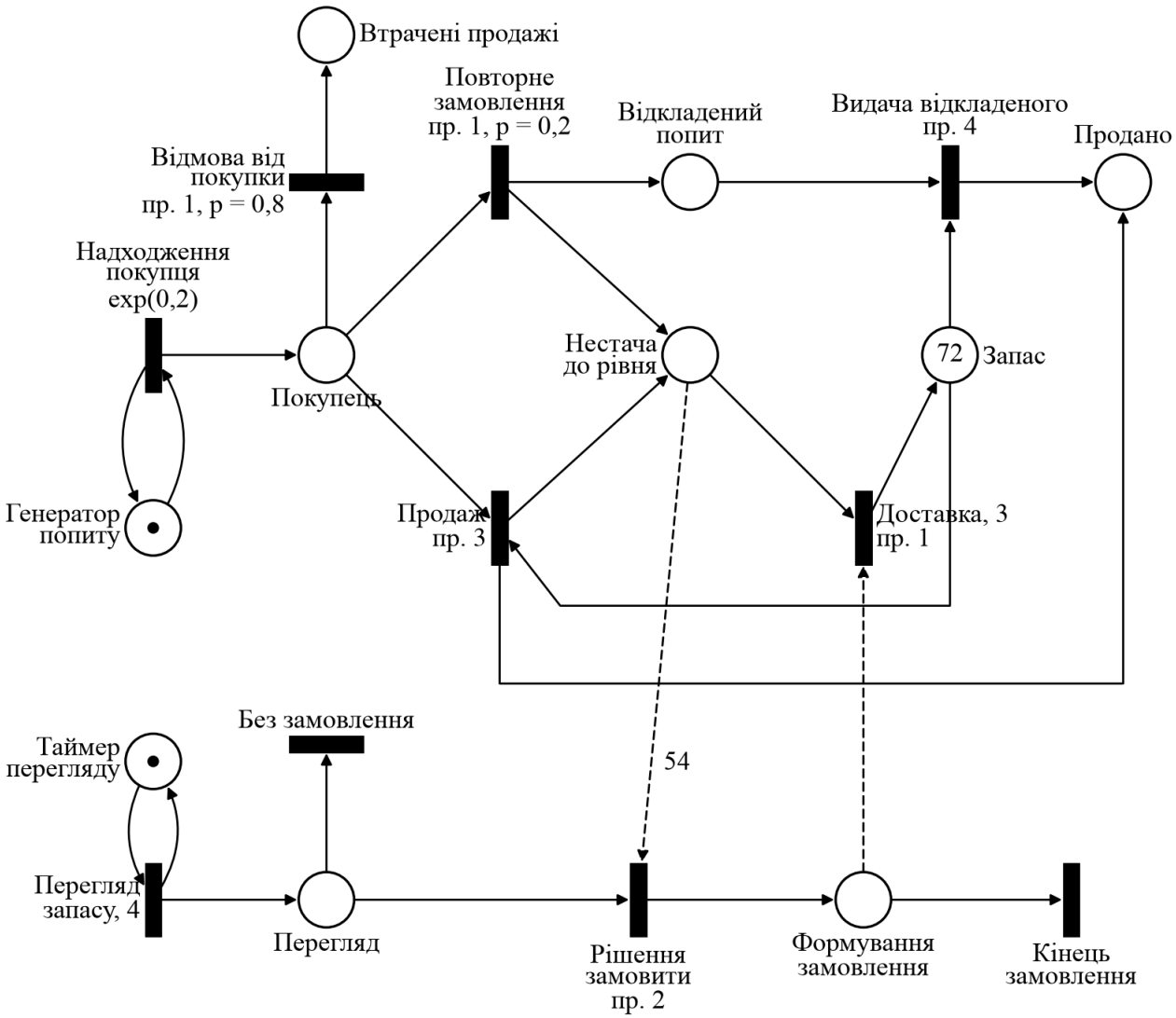


Рисунок 8 – Мережа Петрі системи управління запасами холодильників

Таблиця 15 – Позиції мережі управління запасами

Позиція	M_0	Позиція	M_0
Генератор попиту	1	Втрачені продажі	0
Покупець	0	Нестача до рівня	0
Запас	72	Таймер перегляду	1
Продано	0	Перегляд	0
Відкладений попит	0	Формування замовлення	0

Таблиця 16 – Переходи мережі управління запасами (час у тижнях)

Перехід	Затримка	Пр.	Ймов.	Вхідні дуги	Вихідні дуги
Надходження покупця	exp(0,2)	0	1,0	Генератор попиту	Генератор попиту; Покупець
Продаж	0	3	1,0	Покупець; Запас	Продано; Нестача до рівня
Відмова від покупки	0	1	0,8	Покупець	Втрачені продажі
Повторне замовлення	0	1	0,2	Покупець	Відкладений попит; Нестача до рівня
Видача відкладеного	0	4	1,0	Запас; Відкладений попит	Продано
Перегляд запасу	4	0	1,0	Таймер перегляду	Таймер перегляду; Перегляд
Рішення замовити	0	2	1,0	Перегляд; Нестача до рівня ×54 (інф.)	Формування замовлення
Без замовлення	0	0	1,0	Перегляд	–
Доставка	3	1	1,0	Нестача до рівня; Формування замовлення (інф.)	Запас
Кінець замовлення	0	0	1,0	Формування замовлення	–

Мережу наведено на рис. 8, табл. 15, 16. Розмір замовлення змінний, а кратність дуги мережі Петрі стала, тому замість поточного стану запасу IP у мережі зберігається «Нестача до рівня» $D = 72 - IP$: продаж і повторне замовлення додають маркер у D, доставка партії і видача відкладеного попиту D не змінюють. Умова $IP \le 18$ рівносильна $D \ge 54$ і перевіряється інформаційною дугою кратності 54, яка не забирає маркери. Після рішення замовити перехід «Доставка» (пріоритет 1, інформаційна дуга з позиції «Формування замовлення») у той самий момент спрацьовує D разів – кожен холодильник нестачі стає окремим маркером з затримкою 3 тижні; усі вони виходять з переходу одночасно, тобто приходять однією партією розміром $72 - IP$. Коли D вичерпано, перехід «Кінець замовлення» з пріоритетом 0 забирає маркер «Формування замовлення». Конфлікт «Відмова від покупки» / «Повторне замовлення» розв’язується за ймовірностями 0,8 і 0,2, а «Продаж» має вищий пріоритет і спрацьовує, якщо є запас.

Трасування навколо першого замовлення:

```
t = 11.8371
  вихід з переходів: Продаж
  маркування: Продано=72, Нестача до рівня=72
  у переходах: Надходження покупця=1 (вихід 12.461), Перегляд запасу=1 (вихід 12.000)

t = 12.0000
  вихід з переходів: Перегляд запасу
  вхід у переходи:   Рішення замовити, Перегляд запасу
  маркування: Продано=72, Нестача до рівня=72
  у переходах: Надходження покупця=1 (вихід 12.461), Перегляд запасу=1 (вихід 16.000), Рішення
замовити=1 (вихід 12.000)

t = 12.0000
  вихід з переходів: Рішення замовити
  вхід у переходи:   Доставка ×72, Кінець замовлення
  маркування: Продано=72
  у переходах: Надходження покупця=1 (вихід 12.461), Перегляд запасу=1 (вихід 16.000),
Доставка=72 (вихід 15.000), Кінець замовлення=1 (вихід 12.000)

t = 12.0000
  вихід з переходів: Кінець замовлення
  маркування: Продано=72
  у переходах: Надходження покупця=1 (вихід 12.461), Перегляд запасу=1 (вихід 16.000),
Доставка=72 (вихід 15.000)

t = 14.6913
  вихід з переходів: Відмова від покупки
  маркування: Продано=72, Відкладений попит=1, Втрачені продажі=7, Нестача до рівня=1
  у переходах: Надходження покупця=1 (вихід 15.063), Перегляд запасу=1 (вихід 16.000),
Доставка=72 (вихід 15.000)

t = 15.0000
  вихід з переходів: Доставка ×72
  вхід у переходи:   Видача відкладеного
  маркування: Запас=71, Продано=72, Втрачені продажі=7, Нестача до рівня=1
  у переходах: Надходження покупця=1 (вихід 15.063), Видача відкладеного=1 (вихід 15.000),
Перегляд запасу=1 (вихід 16.000)
```

На 12-му тижні запас вичерпано (D = 72), тож замовляється 72 холодильники – 72 маркери одночасно входять у перехід «Доставка». До приходу партії на 15-му тижні 7 продажів втрачено і один покупець оформив повторне замовлення; він першим отримує холодильник з партії.

Таблиця 17 – Результати моделювання системи управління запасами

Показник	**Значення**
середня кількість холодильників у запасі	28,8412 ± 0,0242
середній час між втраченими продажами, тижні	2,3400 ± 0,0231
втрачених продажів за тиждень	0,4270 ± 0,0042
частка втрачених продажів	0,0854 ± 0,0008
частка відкладеного попиту	0,0213 ± 0,0003
середній невдоволений попит	0,1348 ± 0,0024
частка часу без запасу	0,1065 ± 0,0009
середній поточний стан запасу	42,4201 ± 0,0275
середній розмір замовлення	63,3888 ± 0,0778
середній інтервал між замовленнями, тижні	13,8542 ± 0,0300

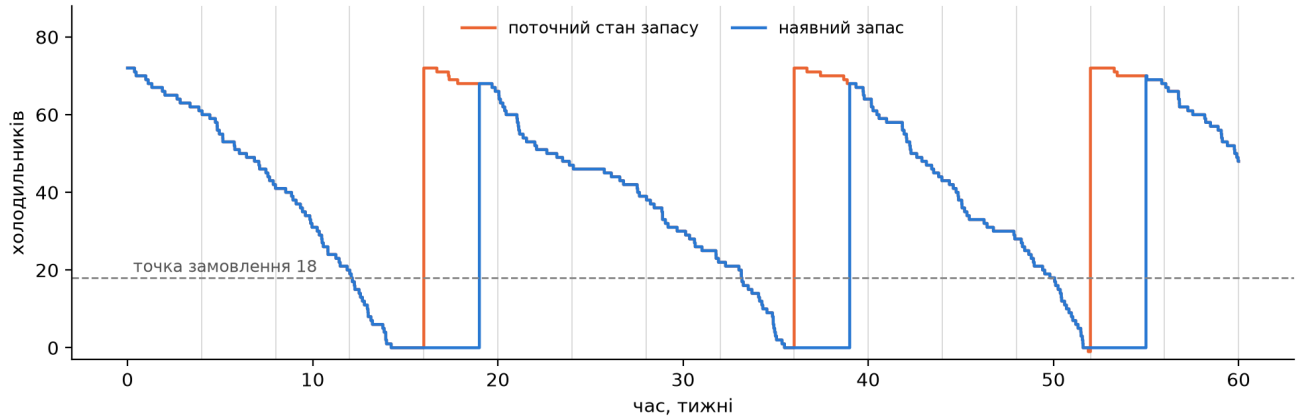


Рисунок 9 – Наявний і поточний стан запасу за перші 60 тижнів

Моделювання сталого режиму: 100 прогонів по 5000 тижнів, розгін 52 тижні (початковий запас 72 холодильники). Середня кількість холодильників у запасі – 28,84 ± 0,02, середній час між продажами, що не здійснилися, – 2,340 ± 0,023 тижня (табл. 17).

Типовий цикл видно на рис. 9: за 4 тижні продається ≈ 20 холодильників, тож поточний стан запасу проходить значення ≈ 72, 52, 32 і на третьому перегляді (≈ 12) опускається нижче точки замовлення; наявного запасу вистачає приблизно на 14,4 тижня, а партія приходить через 15 тижнів, тому наприкінці майже кожного циклу запас вичерпується (частка часу без запасу 10,7 %). Середній інтервал між замовленнями 13,85 тижня: у частині циклів попит за 12 тижнів менший за 54, і замовлення робиться лише на четвертому перегляді. Тому втрачені продажі групуються в кінці циклів: медіана інтервалу між ними 0,185 тижня при середньому 2,34, коефіцієнт варіації 2,70, 87,3 % інтервалів коротші за 4 тижні. Втрачається 8,54 % попиту.

Верифікація моделі:

- після кожної з 53635 подій окремого прогону перевірено P-інваріант: поточний стан запасу + нестача до рівня = 72 (з урахуванням маркера всередині миттєвого переходу «Продаж»); порушень немає;
- інтенсивність попиту 4,9982 покупця за тиждень відповідає заданій 5, а продано 4,5712 + втрачено 0,4270 = 4,9982 – кожен покупець або купує холодильник (зокрема після повторного замовлення), або йде;
- середній інтервал між втраченими продажами узгоджується з оберненою інтенсивністю втрачених продажів 1 / 0,4270 ≈ 2,342 тижня;

- та сама політика управління запасами змодельована незалежною подієвою програмою без мережі Петрі (функція `direct`, 100 прогонів з іншими зернами): запас 28,824 ± 0,026, інтервал між втраченими продажами 2,337 ± 0,024 тижня – збігається з результатами мережі Петрі в межах довірчих інтервалів.

Висновок

У роботі реалізовано алгоритм імітації стохастичної часової мережі Петрі з пріоритетами та ймовірностями спрацювання переходів, багатоканальними переходами, інформаційними дугами та кратностями дуг і побудовано мережі Петрі для чотирьох дискретно-подійних систем. Затримки переходів моделюють тривалість дій, переходи з нульовою затримкою і пріоритетами – миттєві рішення (вибір вільного пристрою, перевага мікроавтобуса А, вибір найближчого робота), ймовірності – випадкову поведінку покупців, позиції з кількома маркерами – обмежені ресурси (місця на верстаті, місця в черзі, вільні місця в мікроавтобусі). Маркери зберігають момент створення, що дозволило вимірювати розподіл часу перебування вимог; середні значення перевірено формулою Літтла, а кожну мережу – P-інваріантами після кожної події.

Конвеєрна система обробляє весь потік деталей (4 за хвилину); середня кількість зайнятих пристроїв дорівнює 4, завантаження спадає вздовж конвеєра від 0,89 до 0,69. Середній час перебування деталі 9,62 хв, але розподіл сильно несиметричний через повернення деталей, які застали всі пристрої зайнятими (медіана 4,5 хв, 95 % деталей – до 32 хв).

Для гнучкої виробничої системи кращим є закріплення по одному роботу за кожним шляхом: пропускна здатність 75,9 проти 72,1 деталі за годину, а за меншого навантаження – менший час перебування деталі. Обидва варіанти не встигають за потоком у 90 деталей за годину; вузьким місцем є верстат B2, місце якого зайняте також під час переміщення деталі і очікування робота, а вибір найближчого робота збільшує холостий хід у момент, коли місце на B2 вже зарезервоване.

Маршрут мікроавтобусів перевантажений за будь-якого $n \leq 25$: середній час очікування в черзі монотонно зменшується зі зростанням n і мінімальний при $n = 25$ (26,9 хв), а виручка за 10-годинний день при цьому становить −7374 грн, оскільки втрати від пасажирів, які йдуть на інший маршрут, перевищують дохід.

У системі управління запасами середня кількість холодильників у запасі становить 28,8, а середній час між продажами, що не здійснилися, – 2,34 тижня; втрачені продажі зосереджені наприкінці циклів замовлення, коли запас вичерпано, а партія ще не надійшла. Змінний розмір замовлення в мережі Петрі зі сталими кратностями дуг реалізовано позицією нестачі до контрольного рівня та інформаційною дугою кратності 54.